

CubeLibrary 源代码

魔方求解器 / Cube Explorer 复刻版 ## 文件列表 - CubeLibrary.py — 主程序 - cl_core.py — 求解引擎 - cl_search.pyx — Cython 搜索核心 - cltimer.py — 计时器 - cl_parser.py — 解析器 - completer.py — 补全+并行求解 - locales.py — 多语言 - setup.py — 编译脚本 — ## 源代码 ### CubeLibrary.py > CubeLibrary.py > 803 行

```
import os
import sys
import json
import queue
import random
import threading
import tkinter as tk
import ttkbootstrap as ttk
from ttkbootstrap.constants import *
from ttkbootstrap.dialogs.colorchooser import ColorChooserDialog
from ttkbootstrap.toast import ToastNotification
import cl_core
import completer
from cltimer import CLTimerTab
from cl_parser import AdvancedParser
import locales
from locales import tr

def get_base_dir():
    return os.path.dirname(os.path.abspath(__file__))

CONFIG_PATH = os.path.join(get_base_dir(), "cl_config.json")

def load_config():
    if os.path.exists(CONFIG_PATH):
        try:
            with open(CONFIG_PATH, 'r', encoding='utf-8') as f:
                return json.load(f)
        except:
            pass
    return {"lang": "en", "theme": "cosmo"}

def save_config(config):
    with open(CONFIG_PATH, 'w', encoding='utf-8') as f:
        json.dump(config, f)

app_config = load_config()
locales.CURRENT_LANG = app_config.get("lang", "zh")
```

```

class CubeExplorerClone(ttk.Window):
    def __init__(self):
        super().__init__(themename=app_config.get("theme", "cosmo"), iconphoto=os.path.join(get_base_dir(), "icon.png"))
        self.title(tr("title"))
        self.geometry("1200x800")

        self.colors = ["#FFFFFF", "#FFFF00", "#008800", "#0000FF", "#FF8800", "#FF0000"]
        self.current_color_idx = 0

        self.facelet_colors = ["#CCCCCC"] * 54
        self.facelet_states = [0] * 54

        self.def_col = {
            'U': self.colors[0], 'D': self.colors[1],
            'F': self.colors[2], 'B': self.colors[3],
            'L': self.colors[4], 'R': self.colors[5]
        }
        self.center_indices = [4, 13, 22, 31, 40, 49]

        C = self.def_col
        self.std_corners = [
            (C['U'], C['R'], C['F']), (C['U'], C['F'], C['L']), (C['U'], C['L'], C['B']), (C['U'], C['B'], C['R']),
            (C['D'], C['F'], C['R']), (C['D'], C['L'], C['F']), (C['D'], C['B'], C['L']), (C['D'], C['R'], C['B'])
        ]
        self.std_edges = [
            (C['U'], C['R']), (C['U'], C['F']), (C['U'], C['L']), (C['U'], C['B']),
            (C['D'], C['R']), (C['D'], C['F']), (C['D'], C['L']), (C['D'], C['B']),
            (C['F'], C['R']), (C['F'], C['L']), (C['B'], C['L']), (C['B'], C['R'])
        ]

        self.corner_indices = [(8, 9, 20), (6, 18, 38), (0, 36, 47), (2, 45, 11), (29, 26, 15), (27, 44, 24), (33, 53, 42), (35, 17, 51)]
        self.edge_indices = [(5, 10), (7, 19), (3, 37), (1, 46), (32, 16), (28, 25), (30, 43), (34, 52), (23, 12), (21, 41), (50, 39), (48, 14)]

        self._build_permutation_matrices()

        self.poly_ids = []
        self.overlay_ids = [None] * 54
        self.maneuver_var = tk.StringVar()

```

```

self.autofix_var = tk.BooleanVar(value=True)
self.search_mode = tk.StringVar(value="twophase")
self.infinite_mode_var = tk.BooleanVar(value=False)
self.solver_threads = tk.IntVar(value=app_config.get("threads",
0))

self._create_menu()
self._create_layout()
self._reset_clean()

self.log_text.insert(tk.END, tr("wait_task"))
threading.Thread(target=cl_core.init_engine, daemon=True).start
()

def _create_menu(self):
    self.menubar = tk.Menu(self)
    self.config(menu=self.menubar)

    self.menu_tools = tk.Menu(self.menubar, tearoff=0)
    self.menubar.add_cascade(label=tr("menu_tools"), menu=self.menu
_tools)
    self.menu_tools.add_command(label=tr("menu_wca"), command=self.
_generate_wca_scramble)

    self.menu_lang = tk.Menu(self.menubar, tearoff=0)
    self.menubar.add_cascade(label=tr("menu_lang"), menu=self.menu_
lang)
    self.menu_lang.add_command(label="English", command=lambda: sel
f._switch_language("en"))
    self.menu_lang.add_command(label="简体中文", command=lambda: sel
f._switch_language("zh"))
    self.menu_lang.add_command(label="繁體中文", command=lambda: sel
f._switch_language("zh_tw"))
    self.menu_lang.add_command(label="Polski", command=lambda: sel
f._switch_language("pl"))

    self.menu_theme = tk.Menu(self.menubar, tearoff=0)
    self.menubar.add_cascade(label=tr("menu_theme"), menu=self.menu
_theme)
    for t in self.style.theme_names():
        self.menu_theme.add_command(label=t.capitalize(), command=l
ambda theme=t: self._switch_theme(theme))

    self.menu_threads = tk.Menu(self.menubar, tearoff=0)
    self.menubar.add_cascade(label=tr("menu_threads"), menu=self.me
nu_threads)
    self._rebuild_threads_menu()

def _switch_language(self, lang):

```

```

    if locales.CURRENT_LANG == lang: return
    locales.CURRENT_LANG = lang
    app_config["lang"] = lang
    save_config(app_config)
    self._update_ui_strings()

def _switch_theme(self, theme_name):
    self.style.theme_use(theme_name)
    app_config["theme"] = theme_name
    save_config(app_config)

def _rebuild_threads_menu(self):
    self.menu_threads.delete(0, tk.END)
    import os
    cpu = os.cpu_count() or 4
    options = [("Auto ({}).format(cpu), 0)]
    for n in [1, 2, 4, 8, 12, 16]:
        if n <= cpu * 2:
            options.append((str(n), n))
    cur = self.solver_threads.get()
    for label, val in options:
        self.menu_threads.add_radiobutton(
            label=label, value=val, variable=self.solver_threads,
            command=lambda v=val: self._set_threads(v))

def _set_threads(self, n):
    self.solver_threads.set(n)
    app_config["threads"] = n
    save_config(app_config)

def _update_ui_strings(self):
    self.title(tr("title"))

    self.menubar.entryconfig(0, label=tr("menu_tools"))
    self.menubar.entryconfig(1, label=tr("menu_lang"))
    self.menubar.entryconfig(2, label=tr("menu_theme"))
    self.menubar.entryconfig(3, label=tr("menu_threads"))
    self._rebuild_threads_menu()
    self.menu_tools.entryconfig(0, label=tr("menu_wca"))

    self.notebook.tab(self.tab_facelet, text=tr("tab_facelet"))
    self.chk_autofix.config(text=tr("autofix"))
    self.lbl_selected_color.config(text=tr("lbl_selected_color"))
    self.btn_customize.config(text=tr("btn_customize"))

    self.grp_solve.config(text=tr("grp_solver"))
    self.rad_twophase.config(text=tr("rad_twophase"))
    self.rad_optimal.config(text=tr("rad_optimal"))
    self.chk_infinite.config(text=tr("chk_infinite"))

```

```

self.btn_solve.config(text=tr("btn_solve"))

self.grp_apply.config(text=tr("grp_apply"))
self.grp_reset.config(text=tr("grp_reset"))
self.btn_export.config(text=tr("btn_export"))
self.btn_empty.config(text=tr("btn_empty"))
self.btn_clean.config(text=tr("btn_clean"))

self.grp_maneuver.config(text=tr("grp_maneuver"))
self.btn_apply.config(text=tr("btn_apply"))
self.btn_clear.config(text=tr("btn_clear"))

self.notebook.tab(self.tab_timer, text=tr("tab_timer"))
self.tab_timer.update_strings()

def _build_permutation_matrices(self):
    raw_perms = {
        'U': 'U3,U6,U9,U2,U5,U8,U1,U4,U7,F1,F2,F3,R4,R5,R6,R7,R8,R9,L1,L2,L3,F4,F5,F6,F7,F8,F9,D1,D2,D3,D4,D5,D6,D7,D8,D9,B1,B2,B3,L4,L5,L6,L7,L8,L9,R1,R2,R3,B4,B5,B6,B7,B8,B9',
        'R': 'U1,U2,B7,U4,U5,B4,U7,U8,B1,R3,R6,R9,R2,R5,R8,R1,R4,R7,F1,F2,U3,F4,F5,U6,F7,F8,U9,D1,D2,F3,D4,D5,F6,D7,D8,F9,L1,L2,L3,L4,L5,L6,L7,L8,L9,D9,B2,B3,D6,B5,B6,D3,B8,B9',
        'F': 'U1,U2,U3,U4,U5,U6,R1,R4,R7,D3,R2,R3,D2,R5,R6,D1,R8,R9,F3,F6,F9,F2,F5,F8,F1,F4,F7,L3,L6,L9,D4,D5,D6,D7,D8,D9,L1,L2,U9,L4,L5,U8,L7,L8,U7,B1,B2,B3,B4,B5,B6,B7,B8,B9',
        'D': 'U1,U2,U3,U4,U5,U6,U7,U8,U9,R1,R2,R3,R4,R5,R6,B7,B8,B9,F1,F2,F3,F4,F5,F6,R7,R8,R9,D3,D6,D9,D2,D5,D8,D1,D4,D7,L1,L2,L3,L4,L5,L6,F7,F8,F9,B1,B2,B3,B4,B5,B6,L7,L8,L9',
        'L': 'F1,U2,U3,F4,U5,U6,F7,U8,U9,R1,R2,R3,R4,R5,R6,R7,R8,R9,D1,F2,F3,D4,F5,F6,D7,F8,F9,B9,D2,D3,B6,D5,D6,B3,D8,D9,L3,L6,L9,L2,L5,L8,L1,L4,L7,B1,B2,U7,B4,B5,U4,B7,B8,U1',
        'B': 'L7,L4,L1,U4,U5,U6,U7,U8,U9,R1,R2,U1,R4,R5,U2,R7,R8,U3,F1,F2,F3,F4,F5,F6,F7,F8,F9,D1,D2,D3,D4,D5,D6,R9,R6,R3,D7,L2,L3,D8,L5,L6,D9,L8,L9,B3,B6,B9,B2,B5,B8,B1,B4,B7',
        'x': 'B9,B8,B7,B6,B5,B4,B3,B2,B1,R3,R6,R9,R2,R5,R8,R1,R4,R7,U1,U2,U3,U4,U5,U6,U7,U8,U9,F1,F2,F3,F4,F5,F6,F7,F8,F9,L7,L4,L1,L8,L5,L2,L9,L6,L3,D9,D8,D7,D6,D5,D4,D3,D2,D1',
        'y': 'U3,U6,U9,U2,U5,U8,U1,U4,U7,F1,F2,F3,F4,F5,F6,F7,F8,F9,L1,L2,L3,L4,L5,L6,L7,L8,L9,D7,D4,D1,D8,D5,D2,D9,D6,D3,B1,B2,B3,B4,B5,B6,B7,B8,B9,R1,R2,R3,R4,R5,R6,R7,R8,R9',
        'z': 'R3,R6,R9,R2,R5,R8,R1,R4,R7,D3,D6,D9,D2,D5,D8,D1,D4,D7,F3,F6,F9,F2,F5,F8,F1,F4,F7,L3,L6,L9,L2,L5,L8,L1,L4,L7,U3,U6,U9,U2,U5,U8,U1,U4,U7,B7,B4,B1,B8,B5,B2,B9,B6,B3',
        'E': 'U1,U2,U3,U4,U5,U6,U7,U8,U9,R1,R2,R3,B4,B5,B6,R7,R8,R9,F1,F2,F3,R4,R5,R6,F7,F8,F9,D1,D2,D3,D4,D5,D6,D7,D8,D9,L1,L2,L3,F4,F5,F6,L7,L8,L9,B1,B2,B3,L4,L5,L6,B7,B8,B9',
        'M': 'U1,F2,U3,U4,F5,U6,U7,F8,U9,R1,R2,R3,R4,R5,R6,R7,R8,R9,F1,D2,F3,F4,D5,F6,F7,D8,F9,D1,B8,D3,D4,B5,D6,D7,B2,D9,L1,L2,L3,L4,L5,

```

```

L6,L7,L8,L9,B1,U8,B3,B4,U5,B6,B7,U2,B9',
    'S': 'U1,U2,U3,R2,R5,R8,U7,U8,U9,R1,D6,R3,R4,D5,R6,R7,D4,R
9,F1,F2,F3,F4,F5,F6,F7,F8,F9,D1,D2,D3,L2,L5,L8,D7,D8,D9,L1,U6,L3,L4,U5,
L6,L7,U4,L9,B1,B2,B3,B4,B5,B6,B7,B8,B9'
    }
    self.moves_perm = {}
    faces = ['U', 'R', 'F', 'D', 'L', 'B']
    for move_name, str_arr in raw_perms.items():
        arr = []
        for item in str_arr.split(','):
            arr.append(faces.index(item[0]) * 9 + int(item[1]) - 1)
        self.moves_perm[move_name] = arr

def _create_layout(self):
    self.paned = ttk.Panedwindow(self, orient=HORIZONTAL)
    self.paned.pack(fill=BOTH, expand=True, padx=5, pady=5)

    self.left_frame = ttk.Frame(self.paned)
    self.paned.add(self.left_frame, weight=1)
    self.log_text = tk.Text(self.left_frame, font=("Consolas", 11),
bg="#1E1E1E", fg="#4AF626", insertbackground="white", relief=FLAT, pad
x=10, pady=10)
    self.log_text.pack(fill=BOTH, expand=True)

    self.right_frame = ttk.Frame(self.paned)
    self.paned.add(self.right_frame, weight=0)
    self.paned.sashpos(0,0)

    self.notebook = ttk.Notebook(self.right_frame, bootstyle=PRIMAR
Y)
    self.notebook.pack(fill=BOTH, expand=True)
    self.tab_facelet = ttk.Frame(self.notebook, padding=10)
    self.notebook.add(self.tab_facelet, text=tr("tab_facelet"))
    self._build_facelet_editor()

    self.tab_timer = CLTimerTab(self.notebook, self, tr)
    self.notebook.add(self.tab_timer, text=tr("tab_timer"))

    self.notebook.bind("<<NotebookTabChanged>>", self._on_tab_chang
ed)

    self.bind("<KeyPress-space>", self._handle_space_press)
    self.bind("<KeyRelease-space>", self._handle_space_release)
    self.is_timer_active = False

def _build_facelet_editor(self):
    top_frame = ttk.Frame(self.tab_facelet)
    top_frame.pack(fill=BOTH, expand=True)

    self.cube_canvas = tk.Canvas(top_frame, bg="#E8E8E8", highlight

```

```

thickness=0)
    self.cube_canvas.pack(fill=BOTH, expand=True)

    self.cube_canvas.bind("<Button-1>", self._handle_left_click)
    self.cube_canvas.bind("<Button-3>", self._handle_right_click)
    self.cube_canvas.bind("<Button-2>", self._handle_right_click)

    self._init_exact_delphi_geometry(self.cube_canvas, l=14, ox=50,
oy=20)

    self.chk_autofix = ttk.Checkbutton(top_frame, text=tr("autofix
"), variable=self.autofix_var, bootstyle="success-round-toggle")
    self.chk_autofix.place(x=20, y=360)

    self.color_box = tk.Canvas(top_frame, width=80, height=80, bg=s
elf.colors[self.current_color_idx], relief=SOLID, borderwidth=2)
    self.color_box.place(x=520, y=160)
    self.color_box.bind("<Button-1>", lambda e: self._cycle_color
())

    self.lbl_selected_color = ttk.Label(top_frame, text=tr("lbl_sel
ected_color"), font=("Arial", 11, "bold"))
    self.lbl_selected_color.place(x=510, y=255)
    self.btn_customize = ttk.Button(top_frame, text=tr("btn_customi
ze"), bootstyle=OUTLINE, command=self._customize_color)
    self.btn_customize.place(x=513, y=285)

    bot_frame = ttk.Frame(self.tab_facelet)
    bot_frame.pack(fill=X, side=BOTTOM, pady=(5,0))

    self.grp_solve = ttk.Labelframe(bot_frame, text=tr("grp_solver
"), padding=10)
    self.grp_solve.grid(row=0, column=0, sticky=NSEW, padx=5)

    self.rad_twophase = ttk.Radiobutton(self.grp_solve, text=tr("ra
d_twophase"), variable=self.search_mode, value="twophase", bootstyle=IN
FO)
    self.rad_twophase.pack(anchor=W, pady=2)
    self.rad_optimal = ttk.Radiobutton(self.grp_solve, text=tr("rad
_optimal"), variable=self.search_mode, value="optimal", bootstyle=INFO)
    self.rad_optimal.pack(anchor=W, pady=2)
    self.chk_infinite = ttk.Checkbutton(self.grp_solve, text=tr("ch
k_infinite"), variable=self.infinite_mode_var, bootstyle="warning-round
-toggle")
    self.chk_infinite.pack(anchor=W, pady=4)

    self.btn_solve = ttk.Button(self.grp_solve, text=tr("btn_solve
"), bootstyle=SUCCESS, command=self._run_solver)
    self.btn_solve.pack(fill=X, pady=4)

```

```

        self.grp_apply = ttk.Labelframe(bot_frame, text=tr("grp_apply
"), padding=10)
        self.grp_apply.grid(row=0, column=1, sticky=NSEW, padx=5)
        move_grid = [['R', 'U', 'F'], ['L', 'D', 'B'], ['M', 'E', 'S'],
['x', 'y', 'z']]
        for r, row_moves in enumerate(move_grid):
            for c, m in enumerate(row_moves):
                btn = ttk.Button(self.grp_apply, text=m, width=4, boots
tyle=SECONDARY)
                btn.config(command=lambda move=m: self._add_and_apply_m
ove(move))
                btn.bind("<ButtonPress-3>", lambda e, btn_ref=btn, move
=m: btn_ref.configure(text=move + ""))
                btn.bind("<ButtonPress-2>", lambda e, btn_ref=btn, move
=m: btn_ref.configure(text=move + ""))
                btn.bind("<ButtonRelease-3>", lambda e, btn_ref=btn, mo
ve=m: self._on_right_click_release(btn_ref, move))
                btn.bind("<ButtonRelease-2>", lambda e, btn_ref=btn, mo
ve=m: self._on_right_click_release(btn_ref, move))
                btn.grid(row=r, column=c, padx=3, pady=2)

        self.grp_reset = ttk.Labelframe(bot_frame, text=tr("grp_reset
"), padding=10)
        self.grp_reset.grid(row=0, column=2, sticky=NSEW, padx=5)
        self.btn_export = ttk.Button(self.grp_reset, text=tr("btn_expor
t"), bootstyle=PRIMARY, command=self._export_kociemba_string)
        self.btn_export.grid(row=0, column=0, columnspan=2, sticky=EW,
pady=(0, 5))
        self.btn_empty = ttk.Button(self.grp_reset, text=tr("btn_empty
"), bootstyle=WARNING, command=self._reset_empty)
        self.btn_empty.grid(row=1, column=0, padx=2, pady=2)
        self.btn_clean = ttk.Button(self.grp_reset, text=tr("btn_clean
"), bootstyle=SUCCESS, command=self._reset_clean)
        self.btn_clean.grid(row=1, column=1, padx=2, pady=2)

        self.grp_maneuver = ttk.Labelframe(bot_frame, text=tr("grp_mane
uver"), padding=10)
        self.grp_maneuver.grid(row=1, column=0, columnspan=3, sticky=E
W, padx=5, pady=10)
        ttk.Entry(self.grp_maneuver, textvariable=self.maneuver_var, fo
nt=("Consolas", 12)).pack(side=LEFT, fill=X, expand=True, padx=5)
        self.btn_apply = ttk.Button(self.grp_maneuver, text=tr("btn_app
ly"), bootstyle=SUCCESS, width=10, command=self._apply_maneuver_string)
        self.btn_apply.pack(side=RIGHT, padx=5)
        self.btn_clear = ttk.Button(self.grp_maneuver, text=tr("btn_cle
ar"), bootstyle=DANGER, width=8, command=lambda: self.maneuver_var.set
(""))
        self.btn_clear.pack(side=RIGHT, padx=5)
    def sync_from_timer(self, scramble_str):

```

```

self.notebook.select(self.tab_facelet)
self._reset_clean()
self.maneuver_var.set(scramble_str + " ")
for m in scramble_str.split():
    self._execute_move_notation(m)
self._sync_colors_to_canvas()
self.log_text.insert(tk.END, f"\n[Sync] Scramble synced from Ti
mer:\n>> {scramble_str}\n")
self.log_text.see(tk.END)
def _init_exact_delphi_geometry(self, canvas, l, ox, oy):
    self.poly_ids = []
    for face in range(6):
        for b in range(3):
            for a in range(3):
                if face == 0:
                    x = ox + l*(3*a - 2*b + 15); y = oy + l*(2*b)
                    pts = [x, y, x+3*l, y, x+l, y+2*l, x-2*l, y+2*
1]

                elif face == 1:
                    x = ox + l*(2*a + 18); y = oy + l*(-2*a + 3*b +
6)

                    pts = [x, y, x+2*l, y-2*l, x+2*l, y+l, x, y+3*
1]

                elif face == 2:
                    x = ox + l*(3*a + 9); y = oy + l*(3*b + 6)
                    pts = [x, y, x+3*l, y, x+3*l, y+3*l, x, y+3*l]
                elif face == 3:
                    x = ox + l*(3*a + 9); y = oy + l*(3*b + 15)
                    pts = [x, y, x+3*l, y, x+3*l, y+3*l, x, y+3*l]
                elif face == 4:
                    x = ox + l*(3*a); y = oy + l*(3*b + 6)
                    pts = [x, y, x+3*l, y, x+3*l, y+3*l, x, y+3*l]
                elif face == 5:
                    x = ox + l*(3*a + 24); y = oy + l*(3*b)
                    pts = [x, y, x+3*l, y, x+3*l, y+3*l, x, y+3*l]
            pid = canvas.create_polygon(*pts, outline="#333333
", width=1.5)
            self.poly_ids.append(pid)
def _on_tab_changed(self, event):
    selected_tab = event.widget.select()
    if selected_tab == str(self.tab_timer):
        self.is_timer_active = True
        self.focus_set()
    else:
        self.is_timer_active = False
def _handle_space_press(self, event):
    if self.is_timer_active:
        self.tab_timer.on_space_press(event)
def _handle_space_release(self, event):
    if self.is_timer_active:

```

```

        self.tab_timer.on_space_release(event)
    def _generate_wca_scramble(self):
        self.log_text.insert(tk.END, f"\n{tr('wait_scramble')}\n")
        self.log_text.see(tk.END)

    def _task():
        state = list("UUUUUUUUURRRRRRRRRFFFFFFFFFDDDDDDDDLLLLLLLLLL
BBBBBBBBBB")
        last_face = ''
        for _ in range(40):
            face = random.choice(['U', 'D', 'R', 'L', 'F', 'B'])
            while face == last_face: face = random.choice(['U', 'D
', 'R', 'L', 'F', 'B'])
            last_face = face
            power = random.choice([1, 2, 3])
            perm = self.moves_perm[face]
            for _ in range(power):
                state = [state[perm[i]] for i in range(54)]

        state_str = "".join(state)
        try:
            sol = next(cl_core.solve(state_str, mode="twophase", ma
x_depth=24, stop_flag=lambda: False))
            inv_map = {"U": "U'", "U'": "U", "U2": "U2", "R": "R'", "R'
": "R", "R2": "R2", "F": "F'", "F'": "F", "F2": "F2",
                    "D": "D'", "D'": "D", "D2": "D2", "L": "L'", "L'
": "L", "L2": "L2", "B": "B'", "B'": "B", "B2": "B2"}
            moves = sol.split()
            scramble = " ".join([inv_map[m] for m in reversed(move
s)]) if moves else ""

            self.after(0, lambda: self.log_text.insert(tk.END, f">>
WCA Scramble: {scramble}\n\n"))
            self.after(0, lambda: self.log_text.see(tk.END))
        except Exception as e:
            self.after(0, lambda: self.log_text.insert(tk.END, f"[E
rror] {e}\n"))

    threading.Thread(target=_task, daemon=True).start()

    def _run_solver(self):
        try:
            state_str = self._extract_state_string()
            mode = self.search_mode.get()
            is_incomplete = "?" in state_str
            is_infinite = self.infinite_mode_var.get()

            if is_infinite:
                self.log_text.insert(tk.END, f"\n[Task] Infinite Search

```

```

        Started ({mode.upper()}...\\n")
        self.log_text.see(tk.END)
        ContinuousSearchDialog(self, state_str, mode, is_incomplete)
    else:
        self.log_text.insert(tk.END, f"\\n[Task] Quick Search (Max 5) Started ({mode.upper()}...\\n")
        self.log_text.see(tk.END)
        self.solve_stop_flag = False
        threading.Thread(target=self._solve_limited_task, args=(state_str, mode, is_incomplete), daemon=True).start()

    except Exception as e:
        self.log_text.insert(tk.END, f"\\n[Error] {e}\\n")
        self.log_text.see(tk.END)

def _solve_limited_task(self, state_str, mode, is_incomplete):
    import time
    t_start = time.time()
    count = 0

    if is_incomplete:
        res_q = queue.Queue()
        worker = threading.Thread(target=completer.solve_incomplete_stream, args=(state_str, res_q, lambda: self.solve_stop_flag, mode, self.solver_threads.get()), daemon=True)
        worker.start()

        while count < 5:
            if self.solve_stop_flag: break
            try:
                msg = res_q.get(timeout=0.1)
                if msg[0] == "DONE": break
                elif msg[0] == "ERROR":
                    self.after(0, lambda m=msg[1]: self.log_text.insert(tk.END, f"\\n[Error] {m}\\n"))
                    break
            else:
                sol = msg[2]
                moves_len = msg[3]
                count += 1
                out_msg = f"    - ({moves_len}f) {sol}\\n"
                self.after(0, lambda m=out_msg: self.log_text.insert(tk.END, m))
                self.after(0, lambda: self.log_text.see(tk.END))

        except queue.Empty:
            continue
        self.solve_stop_flag = True

```

```

        else:
            try:
                for sol in cl_core.solve(state_str, mode=mode, stop_flag=
lambda: self.solve_stop_flag):
                    if count >= 5: break
                    count += 1
                    moves_count = len(sol.split()) if sol else 0
                    out_msg = f" - ({moves_count})f) {sol if sol else
'Already solved'}\n"
                    self.after(0, lambda m=out_msg: self.log_text.insert
t(tk.END, m))
                    self.after(0, lambda: self.log_text.see(tk.END))
            except Exception:
                pass
            self.solve_stop_flag = True

        t_end = time.time()
        end_msg = f">> Search finished. Displaying {count} solution(s)
in {t_end - t_start:.3f}s.\n"
        self.after(0, lambda: self.log_text.insert(tk.END, end_msg))
        self.after(0, lambda: self.log_text.see(tk.END))

    def _export_kociemba_string(self):
        try:
            k_string = self._extract_state_string()
            self.log_text.insert(END, f"\n--- Exported State ---\n{k_st
ring}\n")
            self.log_text.see(END)
        except Exception as e:
            self.log_text.insert(END, f"\n[Error] {e}\n")
            self.log_text.see(END)

    def _extract_state_string(self):
        color_to_face = {}
        for face, idx in zip(['U', 'R', 'F', 'D', 'L', 'B'], self.cente
r_indices):
            color_to_face[self.facelet_colors[idx]] = face
            k_string = ""
            for i in range(54):
                color, state = self.facelet_colors[i], self.facelet_states
[i]
                if color == "#CCCCCC": k_string += "?"
                else:
                    face_char = color_to_face.get(color, "?")
                    if state == 1 and face_char != "?": k_string += face_ch
ar.lower()
                    else: k_string += face_char
            return k_string

```

```

def _get_piece_indices(self, fc):
    for c in self.corner_indices:
        if fc in c: return c
    for e in self.edge_indices:
        if fc in e: return e
    return [fc]

def _handle_left_click(self, event):
    items = self.cube_canvas.find_closest(event.x, event.y)
    if not items: return
    poly_id = items[0]
    if poly_id not in self.poly_ids: return

    idx = self.poly_ids.index(poly_id)

    is_shift = (event.state & 0x0001) != 0
    is_ctrl = (event.state & 0x0004) != 0
    if sys.platform == "darwin" and ((event.state & 0x0008) != 0 or
    (event.state & 0x0010) != 0):
        is_ctrl = True

    if idx in self.center_indices:
        picked_color = self.facelet_colors[idx]
        if picked_color in self.colors:
            self.current_color_idx = self.colors.index(picked_color)

            self.color_box.config(bg=picked_color)
            return

    piece_indices = self._get_piece_indices(idx)

    if is_ctrl:
        for i in piece_indices:
            self.facelet_colors[i] = "#CCCCCC"
            self.facelet_states[i] = 0
    elif is_shift:
        self.facelet_colors[idx] = self.colors[self.current_color_idx]

        self.facelet_states[idx] = 1
        self._autofix(idx)
        for i in piece_indices:
            self.facelet_states[i] = 1
    else:
        self.facelet_colors[idx] = self.colors[self.current_color_idx]

        self.facelet_states[idx] = 0
        for i in piece_indices:
            if i != idx and self.facelet_states[i] != 0:
                self.facelet_states[i] = 0

```

```

        if self.autofix_var.get():
            self._autofix(idx)
        self._sync_colors_to_canvas()

def _handle_right_click(self, event):
    items = self.cube_canvas.find_closest(event.x, event.y)
    if not items: return
    poly_id = items[0]
    if poly_id in self.poly_ids:
        idx = self.poly_ids.index(poly_id)
        if idx not in self.center_indices:
            self.facelet_colors[idx] = "#CCCCCC"
            self.facelet_states[idx] = 0
            self._sync_colors_to_canvas()

def _autofix(self, fc):
    piece_indices = None
    is_corner = False
    for c in self.corner_indices:
        if fc in c: piece_indices = c; is_corner = True; break
    if not piece_indices:
        for e in self.edge_indices:
            if fc in e: piece_indices = e; break
    if not piece_indices: return

    curr_colors = [self.facelet_colors[i] for i in piece_indices]
    empty_count = curr_colors.count("#CCCCCC")
    valid_pieces = self.std_corners if is_corner else self.std_edge

s
    matches = []
    for vp in valid_pieces:
        n = len(vp)
        for shift in range(n):
            shifted_vp = vp[shift:] + vp[:shift]
            compatible = True
            for i in range(n):
                if curr_colors[i] != "#CCCCCC" and curr_colors[i] !=
= shifted_vp[i]:
                    compatible = False; break
            if compatible: matches.append(shifted_vp)

    if len(matches) == 0:
        self.facelet_colors[fc] = "#CCCCCC"
    elif len(matches) == 1 and empty_count == 1:
        match = matches[0]
        for i in range(len(piece_indices)):
            if self.facelet_colors[piece_indices[i]] == "#CCCCCC":
                self.facelet_colors[piece_indices[i]] = match[i]
                self.facelet_states[piece_indices[i]] = 0

```

```

def _on_right_click_release(self, btn, base_move):
    btn.configure(text=base_move)
    self._add_and_apply_move(base_move + "")

def _update_standard_pieces(self):
    self.def_col = {
        'U': self.colors[0], 'D': self.colors[1],
        'F': self.colors[2], 'B': self.colors[3],
        'L': self.colors[4], 'R': self.colors[5]
    }
    C = self.def_col
    self.std_corners = [
        (C['U'], C['R'], C['F']), (C['U'], C['F'], C['L']), (C['U'],
        C['L'], C['B']), (C['U'], C['B'], C['R']),
        (C['D'], C['F'], C['R']), (C['D'], C['L'], C['F']), (C['D'],
        C['B'], C['L']), (C['D'], C['R'], C['B'])
    ]
    self.std_edges = [
        (C['U'], C['R']), (C['U'], C['F']), (C['U'], C['L']), (C['U'],
        C['B']),
        (C['D'], C['R']), (C['D'], C['F']), (C['D'], C['L']), (C['D'],
        C['B']),
        (C['F'], C['R']), (C['F'], C['L']), (C['B'], C['L']), (C['B'],
        C['R'])
    ]

def _customize_color(self):
    curr_color = self.colors[self.current_color_idx]
    cd = ColorChooserDialog(initialcolor=curr_color, title=tr("btn_
customize"))
    cd.show()
    colors = cd.result
    if colors:
        hex_color = colors.hex
        self.colors[self.current_color_idx] = hex_color
        self.color_box.config(bg=hex_color)
        self._update_standard_pieces()
        for i in range(54):
            if self.facelet_colors[i] == curr_color:
                self.facelet_colors[i] = hex_color
        self._sync_colors_to_canvas()

def _cycle_color(self):
    self.current_color_idx = (self.current_color_idx + 1) % len(self.colors)
    self.color_box.config(bg=self.colors[self.current_color_idx])

def _apply_turn_base(self, move):

```

```

        if move not in self.moves_perm: return
        perm = self.moves_perm[move]
        temp_colors = self.facelet_colors[:]
        temp_states = self.facelet_states[:]
        new_colors = [""] * 54
        new_states = [0] * 54
        for i in range(54):
            new_colors[perm[i]] = temp_colors[i]
            new_states[perm[i]] = temp_states[i]
        self.facelet_colors = new_colors
        self.facelet_states = new_states

    def _execute_move_notation(self, move_str):
        move_str = move_str.strip()
        if not move_str: return
        base_move = move_str[0]
        if base_move not in self.moves_perm: return
        if len(move_str) == 1: self._apply_turn_base(base_move)
        elif len(move_str) >= 2 and move_str[1] == '2':
            self._apply_turn_base(base_move); self._apply_turn_base(bas
e_move)
        elif len(move_str) >= 2 and move_str[1] == "'":
            self._apply_turn_base(base_move); self._apply_turn_base(bas
e_move); self._apply_turn_base(base_move)

    def _apply_maneuver_string(self):
        raw_alg = self.maneuver_var.get()
        try:
            expanded_alg = AdvancedParser.parse(raw_alg)
            for m in expanded_alg.split():
                self._execute_move_notation(m)
            self._sync_colors_to_canvas()
            if expanded_alg != " ".join(raw_alg.split()):
                self.log_text.insert(tk.END, f"> Custom Alg: {raw_alg}\n
Expanded: {expanded_alg}\n")
            else:
                self.log_text.insert(tk.END, f"> Executed: {expanded_al
g}\n")
            self.log_text.see(tk.END)
        except Exception as e:
            self.log_text.insert(tk.END, f"[Error] Parsing notation fai
led: {e}\n")
            self.log_text.see(tk.END)

    def _add_and_apply_move(self, move_str):
        curr = self.maneuver_var.get()
        self.maneuver_var.set(curr + move_str + " ")
        self._execute_move_notation(move_str)
        self._sync_colors_to_canvas()

```

```

def _reset_clean(self):
    faces = ['U', 'R', 'F', 'D', 'L', 'B']
    for i in range(6):
        for j in range(9):
            self.facelet_colors[i*9 + j] = self.def_col[faces[i]]
            self.facelet_states[i*9 + j] = 0
    self.maneuver_var.set("")
    self._sync_colors_to_canvas()

def _reset_empty(self):
    self.facelet_colors = ["#CCCCCC"] * 54
    self.facelet_states = [0] * 54
    self._sync_colors_to_canvas()

def _sync_colors_to_canvas(self):
    for i in range(54):
        self.cube_canvas.itemconfig(self.poly_ids[i], fill=self.facelet_colors[i])
        if self.overlay_ids[i]:
            for oid in self.overlay_ids[i]:
                self.cube_canvas.delete(oid)
            self.overlay_ids[i] = []
        else:
            self.overlay_ids[i] = []

        state = self.facelet_states[i]
        if state in (1, 2):
            c = self.cube_canvas.coords(self.poly_ids[i])
            if state == 1:
                l1 = self.cube_canvas.create_line(c[0], c[1], c[4], c[5], fill="#222222", width=2.5)
                l2 = self.cube_canvas.create_line(c[2], c[3], c[6], c[7], fill="#222222", width=2.5)
                self.overlay_ids[i].extend([l1, l2])
            elif state == 2:
                cx, cy = (c[0]+c[2]+c[4]+c[6])/4, (c[1]+c[3]+c[5]+c[7])/4
                o = self.cube_canvas.create_oval(cx-8, cy-8, cx+8, cy+8, fill="white", outline="#222222", width=2.5)
                self.overlay_ids[i].append(o)

class ContinuousSearchDialog(ttk.Toplevel):
    def __init__(self, parent, state_str, search_mode, is_incomplete):
        super().__init__(parent)
        title_prefix = tr("dlg_inc") if is_incomplete else tr("dlg_com")
        self.title(tr("dlg_title").format(title_prefix, search_mode.upper))

```

```

er()))
    self.geometry("750x450")

    self.state_str = state_str
    self.search_mode = search_mode
    self.is_incomplete = is_incomplete
    self.stop_requested = False
    self.result_queue = queue.Queue(maxsize=20)

    top_frame = ttk.Frame(self, padding=10)
    top_frame.pack(fill=X)
    self.lbl_status = ttk.Label(top_frame, text=tr("dlg_status_sear
ch"), font=("Arial", 11, "bold"))
    self.lbl_status.pack(side=LEFT)
    self.btn_stop = ttk.Button(top_frame, text=tr("dlg_btn_stop"),
bootstyle="danger", command=self.stop_search)
    self.btn_stop.pack(side=RIGHT)

    cols = ("ID", "Length", "Maneuver", "State") if is_incomplete e
lse ("ID", "Length", "Maneuver")
    self.tree = ttk.Treeview(self, columns=cols, show="headings", h
eight=15)
    self.tree.heading("ID", text=tr("dlg_col_id"))
    self.tree.heading("Length", text=tr("dlg_col_len"))
    self.tree.heading("Maneuver", text=tr("dlg_col_sol"))
    self.tree.column("ID", width=50, anchor=CENTER)
    self.tree.column("Length", width=50, anchor=CENTER)
    self.tree.column("Maneuver", width=350)

    if is_incomplete:
        self.tree.heading("State", text=tr("dlg_col_state"))
        self.tree.column("State", width=250)

    self.tree.pack(fill=BOTH, expand=True, padx=10, pady=10)
    self.tree.bind("<Control-c>", self.copy_selected)

    if is_incomplete:
        self.worker = threading.Thread(
            target=completer.solve_incomplete_stream,
            args=(self.state_str, self.result_queue, lambda: self.s
top_requested, self.search_mode, parent.solver_threads.get())
        )
    else:
        self.worker = threading.Thread(target=self._solve_complete_
stream)

    self.worker.daemon = True
    self.worker.start()
    self.after(50, self.process_queue)

```

```

def copy_selected(self, event=None):
    sel = self.tree.selection()
    if sel:
        item = self.tree.item(sel[0])
        maneuver = item['values'][2]
        self.clipboard_clear()
        self.clipboard_append(maneuver)
        ToastNotification(title=tr("copy_title"), message=tr("copy_
msg") + f"\n{maneuver}", duration=2000).show_toast()

def _solve_complete_stream(self):
    count = 0
    try:
        for sol in cl_core.solve(self.state_str, mode=self.search_m
ode, stop_flag=lambda: self.stop_requested):
            if self.stop_requested: break
            count += 1
            moves_len = len(sol.split()) if sol else 0
            while not self.stop_requested:
                try:
                    self.result_queue.put((count, self.state_str, s
ol, moves_len), timeout=0.1)
                    break
                except queue.Full:
                    continue
            except Exception as e:
                if not self.stop_requested:
                    self.result_queue.put(("ERROR", str(e)))

    if not self.stop_requested:
        while not self.stop_requested:
            try:
                self.result_queue.put(("DONE", count), timeout=0.1)
                break
            except queue.Full:
                continue

def stop_search(self):
    self.stop_requested = True
    self.lbl_status.config(text=tr("dlg_aborted"))
    self.btn_stop.config(text=tr("dlg_btn_close"), command=self.des
troy, bootstyle="secondary", state=NORMAL)
    while not self.result_queue.empty():
        try:
            self.result_queue.get_nowait()
        except queue.Empty:
            pass

```

```

def process_queue(self):
    while not self.result_queue.empty():
        try:
            msg = self.result_queue.get_nowait()
        except queue.Empty:
            break

        if msg[0] == "DONE":
            self.lbl_status.config(text=tr("dlg_finished").format(m
sg[1]))
            self.btn_stop.config(text=tr("dlg_btn_close"), command=
self.destroy, bootstyle="secondary", state=NORMAL)
            return
        elif msg[0] == "ERROR":
            self.lbl_status.config(text=f"Error: {msg[1]}")
            return
        else:
            count, comp_state, sol, moves_len = msg
            if self.is_incomplete:
                self.tree.insert("", 0, values=(count, f"{moves_le
n}f", sol, comp_state))
            else:
                self.tree.insert("", 0, values=(count, f"{moves_le
n}f", sol))

            self.lbl_status.config(text=tr("dlg_found").format(coun
t))

    if not self.stop_requested:
        self.after(50, self.process_queue)

if __name__ == "__main__":
    app = CubeExplorerClone()
    app.mainloop()

```

cl_core.py

cl_core.py
316 行

```

# =====
# cl_core.py - Cube Library 求解引擎 (Cython 加速版)
# 合并了: 表构建 + 坐标计算 + 状态解析 + solve() 入口
# 搜索热路由 cl_search.pyx (Cython) 提供 ~30x 加速
# 若 Cython 模块不可用, 自动 fallback 到纯 Python 搜索
# =====

import os, math, threading
import numpy as np

```

尝试导入 Cython 搜索模块

```
try:
    import cl_search as _cy
    _HAS_CYTHON = True
except ImportError:
    _cy = None
    _HAS_CYTHON = False
```

— 常量 —

```
_INIT_LOCK = threading.Lock()
MOVES_STR = ["U", "U2", "U'", "R", "R2", "R'", "F", "F2", "F'",
             "D", "D2", "D'", "L", "L2", "L'", "B", "B2", "B'"]
P2_MOVES = [0, 1, 2, 9, 10, 11, 4, 13, 7, 16]

CORNER_FACELETS = ((8, 9, 20), (6, 18, 38), (0, 36, 47), (2, 45, 11),
                   (29, 26, 15), (27, 44, 24), (33, 53, 42), (35, 17, 51))
EDGE_FACELETS = ((5, 10), (7, 19), (3, 37), (1, 46), (32, 16), (28, 25),
                 (30, 43), (34, 52), (23, 12), (21, 41), (50, 39), (48, 14))
STD_CORNERS = (('U', 'R', 'F'), ('U', 'F', 'L'), ('U', 'L', 'B'), ('U', 'B', 'R'),
               ('D', 'F', 'R'), ('D', 'L', 'F'), ('D', 'B', 'L'), ('D', 'R', 'B'))
STD_EDGES = (('U', 'R'), ('U', 'F'), ('U', 'L'), ('U', 'B'),
             ('D', 'R'), ('D', 'F'), ('D', 'L'), ('D', 'B'),
             ('F', 'R'), ('F', 'L'), ('B', 'L'), ('B', 'R'))

FACT = [math.factorial(i) for i in range(13)]
CNK = [[0]*13 for _ in range(13)]
for _n in range(13):
    for _k in range(_n+1): CNK[_n][_k] = math.comb(_n, _k)

FACE_OF = [m//3 for m in range(18)]
CAN_FOLLOW = [[True]*6 for _ in range(7)]
for _lf in range(6):
    CAN_FOLLOW[_lf+1][_lf] = False
    for _cf in (3, 4, 5):
        if _lf == _cf-3: CAN_FOLLOW[_lf+1][_cf] = False
```

— 基础转动 —

```
cp_U=[3,0,1,2,4,5,6,7];co_U=[0]*8;ep_U=[3,0,1,2,4,5,6,7,8,9,10,11];eo_U
=[0]*12
cp_R=[4,1,2,0,7,5,6,3];co_R=[2,0,0,1,1,0,0,2];ep_R=[8,1,2,3,11,5,6,7,4,
9,10,0];eo_R=[0]*12
cp_F=[1,5,2,3,0,4,6,7];co_F=[1,2,0,0,2,1,0,0];ep_F=[0,9,2,3,4,8,6,7,1,
5,10,11];eo_F=[0,1,0,0,0,1,0,0,1,1,0,0]
cp_D=[0,1,2,3,5,6,7,4];co_D=[0]*8;ep_D=[0,1,2,3,5,6,7,4,8,9,10,11];eo_D
=[0]*12
cp_L=[0,2,6,3,4,1,5,7];co_L=[0,1,2,0,0,2,1,0];ep_L=[0,1,10,3,4,5,9,7,8,
2,6,11];eo_L=[0]*12
cp_B=[0,1,3,7,4,5,2,6];co_B=[0,0,1,2,0,0,2,1];ep_B=[0,1,2,11,4,5,6,10,
8,9,3,7];eo_B=[0,0,0,1,0,0,0,1,0,0,1,1]
```

```

BASE_CUBEIES = [(cp_U,co_U,ep_U,eo_U),(cp_R,co_R,ep_R,eo_R),(cp_F,co_F,e
p_F,eo_F),
                 (cp_D,co_D,ep_D,eo_D),(cp_L,co_L,ep_L,eo_L),(cp_B,co_B,e
p_B,eo_B)]

```

— 坐标函数 (展开优化) —————

```

def multiply_cubies(c1, c2):
    c1_0,c1_1,c1_2,c1_3 = c1; c2_0,c2_1,c2_2,c2_3 = c2
    return (
        [c1_0[c2_0[0]],c1_0[c2_0[1]],c1_0[c2_0[2]],c1_0[c2_0[3]],
        c1_0[c2_0[4]],c1_0[c2_0[5]],c1_0[c2_0[6]],c1_0[c2_0[7]]],
        [(c1_1[c2_0[0]]+c2_1[0])%3,(c1_1[c2_0[1]]+c2_1[1])%3,
        (c1_1[c2_0[2]]+c2_1[2])%3,(c1_1[c2_0[3]]+c2_1[3])%3,
        (c1_1[c2_0[4]]+c2_1[4])%3,(c1_1[c2_0[5]]+c2_1[5])%3,
        (c1_1[c2_0[6]]+c2_1[6])%3,(c1_1[c2_0[7]]+c2_1[7])%3],
        [c1_2[c2_2[0]],c1_2[c2_2[1]],c1_2[c2_2[2]],c1_2[c2_2[3]],
        c1_2[c2_2[4]],c1_2[c2_2[5]],c1_2[c2_2[6]],c1_2[c2_2[7]],
        c1_2[c2_2[8]],c1_2[c2_2[9]],c1_2[c2_2[10]],c1_2[c2_2[11]]],
        [(c1_3[c2_2[0]]+c2_3[0])&1,(c1_3[c2_2[1]]+c2_3[1])&1,
        (c1_3[c2_2[2]]+c2_3[2])&1,(c1_3[c2_2[3]]+c2_3[3])&1,
        (c1_3[c2_2[4]]+c2_3[4])&1,(c1_3[c2_2[5]]+c2_3[5])&1,
        (c1_3[c2_2[6]]+c2_3[6])&1,(c1_3[c2_2[7]]+c2_3[7])&1,
        (c1_3[c2_2[8]]+c2_3[8])&1,(c1_3[c2_2[9]]+c2_3[9])&1,
        (c1_3[c2_2[10]]+c2_3[10])&1,(c1_3[c2_2[11]]+c2_3[11])&1])

def get_twist(co):
    return co[0]*729+co[1]*243+co[2]*81+co[3]*27+co[4]*9+co[5]*3+co[6]
def set_twist(val):
    co,p=[0]*8,0
    for i in range(6,-1,-1): co[i]=val%3; p+=co[i]; val//=3
    co[7]=(3-p%3)%3; return co
def get_flip(eo):
    return (eo[0]<<10)|(eo[1]<<9)|(eo[2]<<8)|(eo[3]<<7)|(eo[4]<<6)|(eo
[5]<<5)|(eo[6]<<4)|(eo[7]<<3)|(eo[8]<<2)|(eo[9]<<1)|eo[10]
def set_flip(val):
    eo,p=[0]*12,0
    for i in range(10,-1,-1): eo[i]=val&1; p+=eo[i]; val>>=1
    eo[11]=(2-p%2)%2; return eo
def get_slice(ep):
    s,k=0,3
    for n in range(11,-1,-1):
        if ep[n]>=8: k-=1;
        else: s+=CNK[n][k]
        if k<0: break
    return s
def set_slice(val):
    ep,k=[-1]*12,3
    for n in range(11,-1,-1):
        if val<CNK[n][k]: ep[n]=11-k; k-=1
        else: val-=CNK[n][k]

```

```

        if k<0: break
    cur=0
    for i in range(12):
        if ep[i]==-1: ep[i]=cur; cur+=1
    return ep
def get_perm8(arr):
    a0,a1,a2,a3,a4,a5,a6,a7=arr[0],arr[1],arr[2],arr[3],arr[4],arr[5],a
rr[6],arr[7]
    return (((a1<a0)+(a2<a0)+(a3<a0)+(a4<a0)+(a5<a0)+(a6<a0)+(a7<a0))*5
040+
            ((a2<a1)+(a3<a1)+(a4<a1)+(a5<a1)+(a6<a1)+(a7<a1))*720+
            ((a3<a2)+(a4<a2)+(a5<a2)+(a6<a2)+(a7<a2))*120+
            ((a4<a3)+(a5<a3)+(a6<a3)+(a7<a3))*24+
            ((a5<a4)+(a6<a4)+(a7<a4))*6+((a6<a5)+(a7<a5))*2+(a7<a6))
def get_perm4(arr):
    a0,a1,a2,a3=arr[0],arr[1],arr[2],arr[3]
    return ((a1<a0)+(a2<a0)+(a3<a0))*6+((a2<a1)+(a3<a1))*2+(a3<a2)
def get_perm(arr):
    n=len(arr)
    if n==8: return get_perm8(arr)
    if n==4: return get_perm4(arr)
    r=0
    for i in range(n):
        c=0; ai=arr[i]
        for j in range(i+1,n):
            if arr[j]<ai: c+=1
        r+=c*FACT[n-1-i]
    return r
def set_perm(val,n):
    arr=[0]*n; avail=list(range(n))
    for i in range(n):
        f=FACT[n-1-i]; idx=val//f; val%=f
        arr[i]=avail[idx]; del avail[idx]
    return arr

```

— *Pruning table builder* —

```

def build_2d_pruning_table(MT1,MT2,N1,N2,num_moves):
    total=N1*N2; prun=np.full(total,-1,dtype=np.int8); prun[0]=0
    front=np.array([0],dtype=np.int32); depth=0; filled=1
    while front.size>0 and filled<total:
        x=front//N2; y=front%N2; nexts=[]
        for m in range(num_moves):
            nidx=MT1[x,m].astype(np.int64)*N2+MT2[y,m]
            mask=prun[nidx]==-1; cand=nidx[mask]
            if cand.size==0: continue
            valid=np.unique(cand); valid=valid[prun[valid]==-1]
            if valid.size>0: prun[valid]=depth+1; filled+=valid.size; n
exts.append(valid)
            if not nexts: break
        front=np.unique(np.concatenate(nexts)); depth+=1

```

```

    return prun

# — parse_state —————
def parse_state(s_str):
    cm={s_str[4]:'U',s_str[13]:'R',s_str[22]:'F',s_str[31]:'D',s_str[40]:'L',s_str[49]:'B'}
    s=[cm.get(c,'?') for c in s_str]
    cp,co,ep,eo=[0]*8,[0]*8,[0]*12,[0]*12
    for i in range(8):
        f=(s[CORNER_FACELETS[i][0]],s[CORNER_FACELETS[i][1]],s[CORNER_FACELETS[i][2]]); fs=set(f)
        for j in range(8):
            if fs==set(STD_CORNERS[j]): cp[i]=j; break
        if f[0] in ('U','D'): co[i]=0
        elif f[1] in ('U','D'): co[i]=1
        else: co[i]=2
    for i in range(12):
        f=(s[EDGE_FACELETS[i][0]],s[EDGE_FACELETS[i][1]]); fs=set(f)
        for j in range(12):
            if fs==set(STD_EDGES[j]): ep[i]=j; break
        if f[0] in ('U','D'): eo[i]=0
        elif f[1] in ('U','D'): eo[i]=1
        elif f[0] in ('F','B'): eo[i]=0
        else: eo[i]=1
    return cp,co,ep,eo

# — Engine init —————
INITIALIZED = False
FULL_CUBIE_MOVES = None

def init_engine():
    global INITIALIZED, FULL_CUBIE_MOVES
    if INITIALIZED: return
    with _INIT_LOCK:
        if INITIALIZED: return
        CUBIE_MOVES = []
        for b in BASE_CUBIES:
            m2=multiply_cubies(b,b); m3=multiply_cubies(m2,b)
            CUBIE_MOVES.extend([b,m2,m3])
        FULL_CUBIE_MOVES = CUBIE_MOVES
        fp = os.path.join(os.path.dirname(os.path.abspath(__file__)), "cubie_tables_cache.npz")
        if os.path.exists(fp):
            with np.load(fp) as d:
                tables = {k: d[k] for k in d.files}
        else:
            twist_move=np.zeros((2187,18),dtype=np.int32)
            for i in range(2187):
                co=set_twist(i)
                for m in range(18): cm=CUBIE_MOVES[m]; twist_move[i,m]=

```

```

get_twist([(co[cm[0][j]]+cm[1][j])%3 for j in range(8)])
    flip_move=np.zeros((2048,18),dtype=np.int32)
    for i in range(2048):
        eo=set_flip(i)
        for m in range(18): cm=CUBIE_MOVES[m]; flip_move[i,m]=g
et_flip([(eo[cm[2][j]]+cm[3][j])%2 for j in range(12)])
    slice_move=np.zeros((495,18),dtype=np.int32)
    for i in range(495):
        ep=set_slice(i)
        for m in range(18): cm=CUBIE_MOVES[m]; slice_move[i,m]=
get_slice([ep[cm[2][j]] for j in range(12)])
    cp_move=np.zeros((40320,18),dtype=np.int32)
    for i in range(40320):
        cp_=set_perm(i,8)
        for m in range(18): cm=CUBIE_MOVES[m]; cp_move[i,m]=get
_perm8([cp_[cm[0][j]] for j in range(8)])
    ep_move=np.zeros((40320,10),dtype=np.int32)
    for i in range(40320):
        ep_=set_perm(i,8)+[8,9,10,11]
        for idx,m in enumerate(P2_MOVES): cm=CUBIE_MOVES[m]; ep
_move[i,idx]=get_perm8([ep_[cm[2][j]] for j in range(12)][8])
    sep_move=np.zeros((24,10),dtype=np.int32)
    for i in range(24):
        ep_=[0,1,2,3,4,5,6,7]+[x+8 for x in set_perm(i,4)]
        for idx,m in enumerate(P2_MOVES): cm=CUBIE_MOVES[m]; se
p_move[i,idx]=get_perm4([x-8 for x in [ep_[cm[2][j]] for j in range(1
2)][8:12]])
    cp_move_p2=cp_move[:,P2_MOVES]
    prun_p1_ts=build_2d_pruning_table(twist_move,slice_move,218
7,495,18)
    prun_p1_fs=build_2d_pruning_table(flip_move,slice_move,204
8,495,18)
    prun_p2_cp_sep=build_2d_pruning_table(cp_move_p2,sep_move,4
0320,24,10)
    prun_p2_ep_sep=build_2d_pruning_table(ep_move,sep_move,4032
0,24,10)
    tables = dict(twist_move=twist_move,flip_move=flip_move,sli
ce_move=slice_move,
                  cp_move_p2=cp_move_p2,ep_move=ep_move,sep_mov
e=sep_move,
                  prun_p1_ts=prun_p1_ts,prun_p1_fs=prun_p1_fs,
                  prun_p2_cp_sep=prun_p2_cp_sep,prun_p2_ep_sep=
prun_p2_ep_sep)
    np.savez_compressed(fp, **tables)
    # 加载到 Cython 或 Python fallback
    if _HAS_CYTHON:
        _cy.load_tables(tables['twist_move'],tables['flip_move'],ta
bles['slice_move'],
                        tables['cp_move_p2'],tables['ep_move'],tabl
es['sep_move'],

```

```

        tables['prun_p1_ts'], tables['prun_p1_fs'],
        tables['prun_p2_cp_sep'], tables['prun_p2_ep
_sep'],
        CUBIE_MOVES)
    else:
        _setup_py_globals(tables)
    INITIALIZED = True

# — Python fallback globals —————
def _setup_py_globals(t):
    global twist_move_l, flip_move_l, slice_move_l, cp_move_p2_l, ep_move_
l, sep_move_l
    global prun_p1_ts_b, prun_p1_fs_b, prun_p2_cp_sep_b, prun_p2_ep_sep_b
    twist_move_l=t['twist_move'].tolist(); flip_move_l=t['flip_move'].t
olist()
    slice_move_l=t['slice_move'].tolist(); cp_move_p2_l=t['cp_move_p2
'].tolist()
    ep_move_l=t['ep_move'].tolist(); sep_move_l=t['sep_move'].tolist()
    prun_p1_ts_b=t['prun_p1_ts'].tobytes(); prun_p1_fs_b=t['prun_p1_fs
'].tobytes()
    prun_p2_cp_sep_b=t['prun_p2_cp_sep'].tobytes(); prun_p2_ep_sep_b=t
['prun_p2_ep_sep'].tobytes()

# — solve() — 统一入口 —————
def solve(state_string, mode="twophase", max_depth=22, stop_flag=None):
    init_engine()
    cp,co,ep,eo = parse_state(state_string)
    twist,flip,slc = get_twist(co),get_flip(eo),get_slice(ep)
    if twist==0 and flip==0 and slc==0 and get_perm8(cp)==0 and get_per
m8(ep[:8])==0 and get_perm4([x-8 for x in ep[8:12]])==0:
        yield ""; return
    if _HAS_CYTHON:
        gen = (_cy.solve_optimal_gen if mode=="optimal" else _cy.solve_
twophase_gen)
        yield from gen(int(twist),int(flip),int(slc),cp,co,ep,eo,max_de
pth,stop_flag)
    else:
        yield from _solve_python(cp,co,ep,eo,int(twist),int(flip),int(s
lc),mode,max_depth,stop_flag)

# — 纯 Python 搜索 (fallback) —————
def _solve_python(cp,co,ep,eo,twist,flip,slc,mode,max_depth,stop_flag):
    found_sols=set(); gmin=[max_depth+1]
    _tw=twist_move_l; _fl=flip_move_l; _sl=slice_move_l; _cp=cp_move_p2_l;
_ep=ep_move_l; _sp=sep_move_l
    _pts=prun_p1_ts_b; _pfs=prun_p1_fs_b; _pcs=prun_p2_cp_sep_b; _pes=prun
_p2_ep_sep_b
    _cm=FULL_CUBIE_MOVES; _p2=P2_MOVES; _fo=FACE_OF; _cf=CAN_FOLLOW; _ms=MO
VES_STR; _mc=multiply_cubies

```

```

_nc=[0]; _sf=stop_flag
class Stop(Exception): pass
def _ck():
    _nc[0]+=1
    if _nc[0]&0xFF==0 and _sf and _sf(): raise Stop()
def sp2(cv,ev,sv,g,bd,lf,pa):
    _ck(); h=max(_pcs[cv*24+sv],_pes[ev*24+sv])
    if g+h>bd: return
    if h==0 and g==bd: yield list(pa); return
    for idx,m in enumerate(_p2):
        cf=_fo[m]
        if not _cf[lf+1][cf]: continue
        pa.append(m); yield from sp2(_cp[cv][idx],_ep[ev][idx],_sp
[sv][idx],g+1,bd,cf,pa); pa.pop()
    def sp1(tw,fl,sl,g,bd,p2tl,lf,pa):
        _ck(); h=max(_pts[tw*495+sl],_pfs[fl*495+sl])
        if g+h>bd: return
        if h==0 and g==bd:
            c=cp,co,ep,eo
            for m in pa: c=_mc(c,_cm[m])
            pc,pe,ps=get_perm8(c[0]),get_perm8(c[2][:8]),get_perm4([x-8
for x in c[2][8:12]])
            ph=max(_pcs[pc*24+ps],_pes[pe*24+ps])
            pbs=[p2tl] if p2tl is not None and p2tl>=ph else ([] if p2t
l is not None else range(ph,gmin[0]-g))
            for pb in pbs:
                found=False
                for p2p in sp2(pc,pe,ps,0,pb,pa[-1]//3 if pa else -1,
[]):
                    ss=" ".join([_ms[m] for m in pa+p2p])
                    if ss not in found_sols:
                        found_sols.add(ss)
                        if p2tl is None and g+pb<gmin[0]: gmin[0]=g+pb
                        yield ss; found=True
                    if p2tl is None and found: break
        for m in range(18):
            cf=_fo[m]
            if not _cf[lf+1][cf]: continue
            pa.append(m); yield from sp1(_tw[tw][m],_fl[fl][m],_sl[sl]
[m],g+1,bd,p2tl,cf,pa); pa.pop()
    try:
        if mode=="optimal":
            am=None
            for tl in range(max_depth+1):
                if _sf and _sf(): break
                if am is not None and tl>am+2: break
                for p1b in range(tl+1):
                    if _sf and _sf(): break
                    for sol in sp1(twist,flip,slc,0,p1b,tl-p1b,-1,[]):
                        if am is None: am=tl

```

```

        yield sol
    else:
        for p1b in range(max_depth+1):
            if _sf and _sf(): break
            if p1b>=gmin[0]: break
            yield from sp1(twist,flip,slc,0,p1b,None,-1,[])
except Stop: pass

```

cl_search.pyx

cl_search.pyx
378 行

```

# cython: language_level=3, boundscheck=False, wraparound=False, cdivision=True
"""

```

cl_search.pyx – Cython IDA* search core for Cube Library.
Thread-safe: 每次调用 malloc 独立的 SState, 不使用全局可变状态。
Compile: python setup.py build_ext --inplace
"""

```

import numpy as np
cimport numpy as cnp
from libc.stdlib cimport malloc, free
from libc.string cimport memcpy, memset
cnp.import_array()

```

```

DEF MD = 30
DEF MS = 256
DEF MP2 = 512

```

```

# — 搜索状态结构体 —————
ctypedef struct SState:
    # move tables (只读指针, 所有线程共享)
    int* twist_move
    int* flip_move
    int* slice_move
    int* cp_move_p2
    int* ep_move
    int* sep_move
    unsigned char* prun_ts
    unsigned char* prun_fs
    unsigned char* prun_cp_sep
    unsigned char* prun_ep_sep
    int cubie[18][40]
    int face_of[18]
    int p2_moves[10]
    bint can_follow[7][6]
    # 搜索可变状态 (每线程独立)
    int scp[32][8]
    int sco[32][8]

```

```

int sep_[32][12]
int seo[32][12]
int p1_path[MD]
int p2_path[MD]
int p2_res[MP2][MD]
int p2_lens[MP2]
int p2_cnt
int sol[MS][MD]
int sol_len[MS]
int sol_cnt
int gmin
int max_sol

```

— 只读共享模板 (load_tables 写入一次, 之后只读) —————

```
ctypedef struct SharedTables:
```

```

    int* twist_move
    int* flip_move
    int* slice_move
    int* cp_move_p2
    int* ep_move
    int* sep_move
    unsigned char* prun_ts
    unsigned char* prun_fs
    unsigned char* prun_cp_sep
    unsigned char* prun_ep_sep
    int cubie[18][40]
    int face_of[18]
    int p2_moves[10]
    bint can_follow[7][6]

```

```
cdef SharedTables _shared
```

```
_np_refs = {} # prevent GC
```

```
_tables_loaded = False
```

```
MOVES_STR = ["U", "U2", "U'", "R", "R2", "R'", "F", "F2", "F'",
              "D", "D2", "D'", "L", "L2", "L'", "B", "B2", "B'"]
```

— helpers (nogil) —————

```
cdef inline int perm8(int* a) noexcept nogil:
```

```

    cdef int a0=a[0], a1=a[1], a2=a[2], a3=a[3], a4=a[4], a5=a[5], a6=a[6], a7
    =a[7]

```

```

    return (((a1<a0)+(a2<a0)+(a3<a0)+(a4<a0)+(a5<a0)+(a6<a0)+(a7<a0))*5
    040+

```

```

        ((a2<a1)+(a3<a1)+(a4<a1)+(a5<a1)+(a6<a1)+(a7<a1))*720+
        ((a3<a2)+(a4<a2)+(a5<a2)+(a6<a2)+(a7<a2))*120+
        ((a4<a3)+(a5<a3)+(a6<a3)+(a7<a3))*24+
        ((a5<a4)+(a6<a4)+(a7<a4))*6+
        ((a6<a5)+(a7<a5))*2+(a7<a6))

```

```

cdef inline int perm4(int* a) noexcept nogil:
    return ((a[1]<a[0])+(a[2]<a[0])+(a[3]<a[0]))*6+((a[2]<a[1])+(a[3]<a
[1]))*2+(a[3]<a[2])

```

```

cdef inline void apply_mv(SState* S, int g, int mv) noexcept nogil:
    cdef int j, idx
    cdef int* cm = S.cubie[mv]
    for j in range(8):
        idx = cm[j]
        S.scp[g+1][j] = S.scp[g][idx]
        S.sco[g+1][j] = (S.sco[g][idx] + cm[8+j]) % 3
    for j in range(12):
        idx = cm[16+j]
        S.sep_[g+1][j] = S.sep_[g][idx]
        S.seo[g+1][j] = (S.seo[g][idx] + cm[28+j]) & 1

```

— Phase 2 (nogil) —————

```

cdef void srch_p2(SState* S, int cpv, int epv, int sepv,
    int g, int bound, int lf) noexcept nogil:
    cdef int h1, h2, h, idx, m, cf, lf1
    h1 = <int>S.prun_cp_sep[cpv*24+sepv]
    h2 = <int>S.prun_ep_sep[epv*24+sepv]
    h = h1 if h1>h2 else h2
    if g+h > bound: return
    if h==0 and g==bound:
        if S.p2_cnt < MP2:
            S.p2_lens[S.p2_cnt] = g
            memcpy(&S.p2_res[S.p2_cnt][0], S.p2_path, g*sizeof(int))
            S.p2_cnt += 1
        return
    lf1 = lf+1
    for idx in range(10):
        m = S.p2_moves[idx]; cf = S.face_of[m]
        if not S.can_follow[lf1][cf]: continue
        S.p2_path[g] = m
        srch_p2(S, S.cp_move_p2[cpv*10+idx], S.ep_move[epv*10+idx],
            S.sep_move[sepv*10+idx], g+1, bound, cf)

```

— Phase 1 – twophase (nogil) —————

```

cdef void srch_p1_tp(SState* S, int tw, int fl, int sl,
    int g, int bound, int lf) noexcept nogil:
    cdef int h1,h2,h,m,cf,lf1,pc,pe,ps,ph1,ph2,ph,p2b,ri,tot
    cdef int sa[4]
    h1 = <int>S.prun_ts[tw*495+sl]; h2 = <int>S.prun_fs[fl*495+sl]
    h = h1 if h1>h2 else h2
    if g+h > bound: return

```

```

    if h==0 and g==bound:
        pc = perm8(S.scp[g]); pe = perm8(S.sep_[g])
        sa[0]=S.sep_[g][8]-8; sa[1]=S.sep_[g][9]-8
        sa[2]=S.sep_[g][10]-8; sa[3]=S.sep_[g][11]-8
        ps = perm4(sa)
        ph1 = <int>S.prun_cp_sep[pc*24+ps]; ph2 = <int>S.prun_ep_sep[pe
*24+ps]
        ph = ph1 if ph1>ph2 else ph2
        for p2b in range(ph, S.gmin-g):
            S.p2_cnt = 0
            srch_p2(S, pc, pe, ps, 0, p2b,
                    S.face_of[S.p1_path[g-1]] if g>0 else -1)
            if S.p2_cnt > 0:
                for ri in range(S.p2_cnt):
                    if S.sol_cnt >= S.max_sol: return
                    tot = g + S.p2_lens[ri]
                    if tot < S.gmin: S.gmin = tot
                    S.sol_len[S.sol_cnt] = tot
                    memcpy(&S.sol[S.sol_cnt][0], S.p1_path, g*sizeof(in
t))
                    memcpy(&S.sol[S.sol_cnt][g], &S.p2_res[ri][0],
                        S.p2_lens[ri]*sizeof(int))
                    S.sol_cnt += 1
                break
            return
    lf1 = lf+1
    for m in range(18):
        cf = S.face_of[m]
        if not S.can_follow[lf1][cf]: continue
        S.p1_path[g] = m; apply_mv(S, g, m)
        srch_p1_tp(S, S.twist_move[tw*18+m], S.flip_move[fl*18+m],
                    S.slice_move[sl*18+m], g+1, bound, cf)
        if S.sol_cnt >= S.max_sol: return

```

— Phase 1 – optimal (nogil) —————

```

cdef void srch_p1_opt(SState* S, int tw, int fl, int sl,
                    int g, int bound, int p2tl, int lf) noexcept nogi
l:
    cdef int h1,h2,h,m,cf,lf1,pc,pe,ps,ph1,ph2,ph,ri,tot
    cdef int sa[4]
    h1 = <int>S.prun_ts[tw*495+sl]; h2 = <int>S.prun_fs[fl*495+sl]
    h = h1 if h1>h2 else h2
    if g+h > bound: return
    if h==0 and g==bound:
        pc = perm8(S.scp[g]); pe = perm8(S.sep_[g])
        sa[0]=S.sep_[g][8]-8; sa[1]=S.sep_[g][9]-8
        sa[2]=S.sep_[g][10]-8; sa[3]=S.sep_[g][11]-8
        ps = perm4(sa)
        ph1 = <int>S.prun_cp_sep[pc*24+ps]; ph2 = <int>S.prun_ep_sep[pe

```

```

*24+ps]
    ph = ph1 if ph1>ph2 else ph2
    if p2tl >= ph:
        S.p2_cnt = 0
        srch_p2(S, pc, pe, ps, 0, p2tl,
                S.face_of[S.p1_path[g-1]] if g>0 else -1)
        for ri in range(S.p2_cnt):
            if S.sol_cnt >= S.max_sol: return
            tot = g + S.p2_lens[ri]
            S.sol_len[S.sol_cnt] = tot
            memcpy(&S.sol[S.sol_cnt][0], S.p1_path, g*sizeof(int))
            memcpy(&S.sol[S.sol_cnt][g], &S.p2_res[ri][0],
                S.p2_lens[ri]*sizeof(int))
            S.sol_cnt += 1
        return
    lf1 = lf+1
    for m in range(18):
        cf = S.face_of[m]
        if not S.can_follow[lf1][cf]: continue
        S.p1_path[g] = m; apply_mv(S, g, m)
        srch_p1_opt(S, S.twist_move[tw*18+m], S.flip_move[fl*18+m],
                    S.slice_move[sl*18+m], g+1, bound, p2tl, cf)
        if S.sol_cnt >= S.max_sol: return

```

— 内部工具: 从 *SharedTables* 初始化一个 *SState* —————

```

cdef SState* _new_state(list cp, list co, list ep, list eo):
    """malloc 一个 SState, 复制只读表指针 + 初始化 cubie 栈."""
    cdef SState* S = <SState*>malloc(sizeof(SState))
    if S == NULL:
        raise MemoryError("Failed to allocate SState")
    # 复制只读共享数据
    S.twist_move = _shared.twist_move
    S.flip_move = _shared.flip_move
    S.slice_move = _shared.slice_move
    S.cp_move_p2 = _shared.cp_move_p2
    S.ep_move = _shared.ep_move
    S.sep_move = _shared.sep_move
    S.prun_ts = _shared.prun_ts
    S.prun_fs = _shared.prun_fs
    S.prun_cp_sep = _shared.prun_cp_sep
    S.prun_ep_sep = _shared.prun_ep_sep
    memcpy(S.cubie, _shared.cubie, sizeof(_shared.cubie))
    memcpy(S.face_of, _shared.face_of, sizeof(_shared.face_of))
    memcpy(S.p2_moves, _shared.p2_moves, sizeof(_shared.p2_moves))
    memcpy(S.can_follow, _shared.can_follow, sizeof(_shared.can_follo
w))
    # 初始化 cubie 栈底
    cdef int j

```

```

    for j in range(8):
        S.scp[0][j] = cp[j]; S.sco[0][j] = co[j]
    for j in range(12):
        S.sep_[0][j] = ep[j]; S.seo[0][j] = eo[j]
    S.sol_cnt = 0
    return S

cdef list _collect(SState* S):
    """从 SState 中收集解字符串列表。"""
    cdef int i, k
    result = []; seen = set()
    for i in range(S.sol_cnt):
        parts = []
        for k in range(S.sol_len[i]):
            parts.append(MOVES_STR[S.sol[i][k]])
        s = " ".join(parts)
        if s not in seen:
            seen.add(s); result.append(s)
    return result

# — Python API (线程安全) —————

def load_tables(twist_move_np, flip_move_np, slice_move_np,
                cp_move_p2_np, ep_move_np, sep_move_np,
                prun_ts_np, prun_fs_np, prun_cp_sep_np, prun_ep_sep_np,
                list cubie_moves_list):
    """加载移动表到只读共享模板（只调用一次）。"""
    global _np_refs, _tables_loaded

    cdef cnp.ndarray tw = np.ascontiguousarray(twist_move_np, dtype=np.
    intc)
    cdef cnp.ndarray fl = np.ascontiguousarray(flip_move_np, dtype=np.i
    ntc)
    cdef cnp.ndarray sl = np.ascontiguousarray(slice_move_np, dtype=np.
    intc)
    cdef cnp.ndarray cp = np.ascontiguousarray(cp_move_p2_np, dtype=np.
    intc)
    cdef cnp.ndarray ep = np.ascontiguousarray(ep_move_np, dtype=np.int
    c)
    cdef cnp.ndarray sp = np.ascontiguousarray(sep_move_np, dtype=np.in
    tc)
    cdef cnp.ndarray pts = np.ascontiguousarray(
        np.asarray(prun_ts_np, dtype=np.int8).view(np.uint8))
    cdef cnp.ndarray pfs = np.ascontiguousarray(
        np.asarray(prun_fs_np, dtype=np.int8).view(np.uint8))
    cdef cnp.ndarray pcs = np.ascontiguousarray(
        np.asarray(prun_cp_sep_np, dtype=np.int8).view(np.uint8))
    cdef cnp.ndarray pes = np.ascontiguousarray(
        np.asarray(prun_ep_sep_np, dtype=np.int8).view(np.uint8))

```

```

_shared.twist_move = <int*>tw.data
_shared.flip_move  = <int*>fl.data
_shared.slice_move = <int*>sl.data
_shared.cp_move_p2 = <int*>cp.data
_shared.ep_move    = <int*>ep.data
_shared.sep_move    = <int*>sp.data
_shared.prun_ts     = <unsigned char*>pts.data
_shared.prun_fs     = <unsigned char*>pfs.data
_shared.prun_cp_sep = <unsigned char*>pcs.data
_shared.prun_ep_sep = <unsigned char*>pes.data

```

```

_np_refs = {'tw':tw, 'fl':fl, 'sl':sl, 'cp':cp, 'ep':ep, 'sp':sp,
            'pts':pts, 'pfs':pfs, 'pcs':pcs, 'pes':pes}

```

```

cdef int i, j, a, b
for i in range(18):
    _shared.face_of[i] = i // 3
    _shared.p2_moves[0]=0;_shared.p2_moves[1]=1;_shared.p2_moves[2]=2
    _shared.p2_moves[3]=9;_shared.p2_moves[4]=10;_shared.p2_moves[5]=11
    _shared.p2_moves[6]=4;_shared.p2_moves[7]=13
    _shared.p2_moves[8]=7;_shared.p2_moves[9]=16
    for a in range(7):
        for b in range(6):
            _shared.can_follow[a][b] = True
    for a in range(6):
        _shared.can_follow[a+1][a] = False
    _shared.can_follow[1][3]=False
    _shared.can_follow[2][4]=False
    _shared.can_follow[3][5]=False
    for i in range(18):
        cm = cubie_moves_list[i]
        for j in range(8):
            _shared.cubie[i][j] = cm[0][j]
            _shared.cubie[i][8+j] = cm[1][j]
        for j in range(12):
            _shared.cubie[i][16+j] = cm[2][j]
            _shared.cubie[i][28+j] = cm[3][j]

```

```

_tables_loaded = True

```

```

def solve_twophase_gen(int twist, int flip, int slc,
                       list cp, list co, list ep, list eo,
                       int max_depth, object stop_callable=None):
    """线程安全的 Two-Phase generator。每次调用独立分配搜索状态。"""
    if not _tables_loaded:
        raise RuntimeError("Tables not loaded. Call load_tables() first.")

```

```

cdef SState* S = _new_state(cp, co, ep, eo)
cdef int p1b, prev, i, k
try:
    S.gmin = max_depth + 1
    seen = set()
    for p1b in range(max_depth + 1):
        if stop_callable is not None and stop_callable():
            break
        if p1b >= S.gmin:
            break
        prev = S.sol_cnt
        S.max_sol = prev + 1
        with nogil:
            srch_p1_tp(S, twist, flip, slc, 0, p1b, -1)
    for i in range(prev, S.sol_cnt):
        parts = []
        for k in range(S.sol_len[i]):
            parts.append(MOVES_STR[S.sol[i][k]])
        s = " ".join(parts)
        if s not in seen:
            seen.add(s)
            yield s
finally:
    free(S)

```

```

def solve_optimal_gen(int twist, int flip, int slc,
                      list cp, list co, list ep, list eo,
                      int max_depth, object stop_callable=None):
    """线程安全的 Optimal generator。每次调用独立分配搜索状态。"""
    if not _tables_loaded:
        raise RuntimeError("Tables not loaded. Call load_tables() first.")

```

```

cdef SState* S = _new_state(cp, co, ep, eo)
cdef int tl, p1b, prev, i, k, abs_min = -1
try:
    seen = set()
    for tl in range(max_depth + 1):
        if stop_callable is not None and stop_callable():
            break
        if abs_min >= 0 and tl > abs_min + 2:
            break
        for p1b in range(tl + 1):
            if stop_callable is not None and stop_callable():
                break
            prev = S.sol_cnt
            S.max_sol = prev + 1

```

```

        with nogil:
            srch_p1_opt(S, twist, flip, slc, 0, p1b, t1 - p1b,
-1)

        for i in range(prev, S.sol_cnt):
            parts = []
            for k in range(S.sol_len[i]):
                parts.append(MOVES_STR[S.sol[i][k]])
            s = " ".join(parts)
            if s not in seen:
                seen.add(s)
                if abs_min < 0:
                    abs_min = t1
                yield s

    finally:
        free(S)

```

cltimer.py

cltimer.py

296 行

```

import time
import random
import tkinter as tk
import ttkbootstrap as ttk
from ttkbootstrap.constants import *
from ttkbootstrap.dialogs import Querybox
from ttkbootstrap.toast import ToastNotification

class CLTimerTab(ttk.Frame):
    def __init__(self, master, app_instance, tr_func):
        super().__init__(master, padding=10)
        self.app = app_instance
        self.tr = tr_func

        self.state = 0
        self.start_time = 0.0
        self.elapsed = 0.0

        self.history = []
        self.current_scramble = ""
        self.viewing_idx = -1

        self.ready_job = None
        self.update_job = None
        self._release_debounce_job = None

        self._create_ui()
        self._generate_new_scramble()

```

```

def _create_ui(self):
    self.left_frame = ttk.Frame(self, width=200)
    self.left_frame.pack(side=LEFT, fill=Y, padx=(0, 10))

    self.right_frame = ttk.Frame(self)
    self.right_frame.pack(side=LEFT, fill=BOTH, expand=True)

    self.lbl_hist = ttk.Label(self.left_frame, text=self.tr("timer_
history"), font=("Arial", 10, "bold"))
    self.lbl_hist.pack(anchor=W, pady=(0, 5))

    self.tree = ttk.Treeview(self.left_frame, columns=("ID", "Time
"), show="headings", selectmode="browse")
    self.tree.heading("ID", text=self.tr("timer_col_id"))
    self.tree.heading("Time", text=self.tr("timer_col_time"))
    self.tree.column("ID", width=40, anchor=CENTER)
    self.tree.column("Time", width=100, anchor=CENTER)
    self.tree.pack(fill=BOTH, expand=True)
    self.tree.bind("<<TreeviewSelect>>", self._on_tree_select)

    top_bar = ttk.Frame(self.right_frame)
    top_bar.pack(fill=X, pady=(10, 20))

    self.lbl_scramble = ttk.Label(
        top_bar, text="", font=("Consolas", 18, "bold"),
        wraplength=600, justify=CENTER
    )
    self.lbl_scramble.pack(side=LEFT, expand=True, fill=X)
    self.lbl_scramble.bind("<Enter>", self._on_scramble_hover)
    self.lbl_scramble.bind("<Leave>", self._on_scramble_leave)
    self.lbl_scramble.bind("<Button-1>", self._copy_scramble)

    # == 新增的同步按钮 ==
    self.btn_sync = ttk.Button(top_bar, text=self.tr("timer_btn_syn
c"), bootstyle="info-outline", command=self._sync_to_editor)
    self.btn_sync.pack(side=RIGHT, padx=5)

    self.btn_custom = ttk.Button(top_bar, text=self.tr("timer_btn_c
ustom"), bootstyle="outline", command=self._input_custom_scramble)
    self.btn_custom.pack(side=RIGHT, padx=5)

    self.lbl_time = ttk.Label(self.right_frame, text="0.00", font=
("Arial", 120, "bold"), bootstyle=DEFAULT)
    self.lbl_time.pack(expand=True)

    bot_frame = ttk.Frame(self.right_frame)
    bot_frame.pack(fill=X, side=BOTTOM, pady=20)

    self.lbl_stats = ttk.Label(bot_frame, text="", font=("Arial", 1

```

```

4))

    self.lbl_stats.pack(side=LEFT, padx=10)

    self.btn_next = ttk.Button(bot_frame, text=self.tr("timer_btn_next"), bootstyle=PRIMARY, command=self._generate_new_scramble)
    self.btn_next.pack(side=RIGHT, padx=5)

    self.btn_del = ttk.Button(bot_frame, text=self.tr("timer_btn_delete"), bootstyle="danger-outline", command=self._delete_record)
    self.btn_del.pack(side=RIGHT, padx=(20, 5))

    self.btn_dnf = ttk.Button(bot_frame, text="DNF", bootstyle="warning-outline", command=lambda: self._toggle_penalty("DNF"))
    self.btn_dnf.pack(side=RIGHT, padx=5)

    self.btn_plus2 = ttk.Button(bot_frame, text="+2", bootstyle="warning-outline", command=lambda: self._toggle_penalty("+2"))
    self.btn_plus2.pack(side=RIGHT, padx=5)

    self.update_stats()

def update_strings(self):
    self.lbl_hist.configure(text=self.tr("timer_history"))
    self.tree.heading("ID", text=self.tr("timer_col_id"))
    self.tree.heading("Time", text=self.tr("timer_col_time"))
    self.btn_custom.configure(text=self.tr("timer_btn_custom"))
    self.btn_sync.configure(text=self.tr("timer_btn_sync"))
    self.btn_next.configure(text=self.tr("timer_btn_next"))
    self.btn_del.configure(text=self.tr("timer_btn_delete"))
    self.update_stats()

def _on_scramble_hover(self, event):
    self.lbl_scramble.configure(cursor="hand2")
    self.lbl_scramble.configure(foreground=ttk.Style().colors.primary)

def _on_scramble_leave(self, event):
    self.lbl_scramble.configure(cursor="")
    # 【修复 BUG】: 改为 fg
    self.lbl_scramble.configure(foreground=ttk.Style().colors.fg)

def _copy_scramble(self, event):
    scramble_text = self.lbl_scramble.cget("text")
    if scramble_text:
        self.clipboard_clear()
        self.clipboard_append(scramble_text)
        ToastNotification(title=self.tr("timer_copy_title"), message=self.tr("timer_copy_msg"), duration=1500).show_toast()

```

```

def _input_custom_scramble(self):
    res = Querybox.get_string(self.tr("timer_custom_prompt"), self.
tr("timer_custom_title"))
    if res and res.strip():
        self.current_scramble = res.strip()
        self._exit_review_mode()

# === [新增: 同步给主程序] ===
def _sync_to_editor(self):
    if hasattr(self.app, "sync_from_timer"):
        # 获取当前屏幕上正在显示的公式 (支持历史回顾态)
        target_scramble = self.lbl_scramble.cget("text")
        self.app.sync_from_timer(target_scramble)

def _generate_new_scramble(self):
    moves = ["U", "D", "R", "L", "F", "B"]
    mods = ["", "'", "2"]
    scramble = []
    last = ""
    for _ in range(21):
        m = random.choice(moves)
        while m == last: m = random.choice(moves)
        last = m
        scramble.append(m + random.choice(mods))
    self.current_scramble = " ".join(scramble)
    self._exit_review_mode()

def _on_tree_select(self, event):
    sel = self.tree.selection()
    if not sel: return
    idx = int(self.tree.item(sel[0])['values'][0]) - 1
    self.viewing_idx = idx
    record = self.history[idx]

    self.lbl_scramble.configure(text=record["scramble"])
    self.lbl_time.configure(text=self.format_record(record), bootst
yle=INFO)

def _exit_review_mode(self):
    self.viewing_idx = -1
    for item in self.tree.selection():
        self.tree.selection_remove(item)

    self.lbl_scramble.configure(text=self.current_scramble)
    if self.history:
        self.lbl_time.configure(text=self.format_record(self.histor
y[-1]), bootstyle=DEFAULT)
    else:
        self.lbl_time.configure(text="0.00", bootstyle=DEFAULT)

```

```

def _refresh_treeview(self):
    for row in self.tree.get_children():
        self.tree.delete(row)
    for i, record in enumerate(self.history):
        t_str = self.format_record(record)
        self.tree.insert("", "end", values=(i + 1, t_str))
    if self.tree.get_children():
        self.tree.yview_moveto(1)

def raw_format(self, seconds):
    if seconds == float('inf'): return "DNF"
    if seconds < 60: return f"{seconds:.2f}"
    mins = int(seconds // 60)
    secs = seconds % 60
    return f"{mins}:{secs:05.2f}"

def get_effective_time(self, record):
    if record["penalty"] == "DNF": return float('inf')
    elif record["penalty"] == "+2": return record["base"] + 2.0
    return record["base"]

def format_record(self, record):
    if record["penalty"] == "DNF": return "DNF"
    elif record["penalty"] == "+2": return self.raw_format(record["base"] + 2.0) + "+"
    return self.raw_format(record["base"])

def calculate_wca_average(self, n):
    if len(self.history) < n: return "--"
    effective_times = [self.get_effective_time(r) for r in self.history[-n:]]
    effective_times.sort()
    trimmed = effective_times[1:-1]
    if float('inf') in trimmed: return "DNF"
    return self.raw_format(sum(trimmed) / len(trimmed))

def update_stats(self):
    ao5 = self.calculate_wca_average(5)
    ao12 = self.calculate_wca_average(12)
    stats_text = self.tr("timer_stats")
    if stats_text == "timer_stats":
        stats_text = "Solves: {0} | Ao5: {1} | Ao12: {2}"
    self.lbl_stats.configure(text=stats_text.format(len(self.history), ao5, ao12))

def _get_target_idx(self):
    if not self.history: return -1
    return self.viewing_idx if self.viewing_idx != -1 else len(self.history)

```

```

f.history) - 1

def _delete_record(self):
    idx = self._get_target_idx()
    if idx == -1: return
    self.history.pop(idx)
    self.update_stats()
    self._refresh_treeview()
    self._exit_review_mode()

def _toggle_penalty(self, p_type):
    idx = self._get_target_idx()
    if idx == -1: return

    record = self.history[idx]
    record["penalty"] = "" if record["penalty"] == p_type else p_ty
    pe

    style = INFO if self.viewing_idx != -1 else DEFAULT
    self.lbl_time.configure(text=self.format_record(record), bootst
yle=style)

    self.update_stats()
    self._refresh_treeview()

    if self.viewing_idx != -1:
        items = self.tree.get_children()
        if items and idx < len(items):
            self.tree.selection_set(items[idx])

def on_space_press(self, event):
    if self._release_debounce_job is not None:
        self.after_cancel(self._release_debounce_job)
        self._release_debounce_job = None
        return

    if self.viewing_idx != -1:
        self._exit_review_mode()

    if self.state == 0:
        self.state = 1
        self.lbl_time.configure(bootstyle=DANGER)
        if self.ready_job is not None: self.after_cancel(self.ready
_job)

        self.ready_job = self.after(300, self._set_ready)

    elif self.state == 3:
        self.state = 0
        self.elapsed = time.time() - self.start_time

```

```

        if self.update_job is not None: self.after_cancel(self.update_job)

        new_record = {"base": self.elapsed, "penalty": "", "scramble": self.current_scramble}
        self.history.append(new_record)

        self.lbl_time.configure(text=self.format_record(new_record))

    def _set_ready(self):
        if self.state == 1:
            self.state = 2
            self.lbl_time.configure bootstyle=SUCCESS
            self.lbl_time.configure(text="0.00")

    def on_space_release(self, event):
        if self._release_debounce_job is not None:
            self.after_cancel(self._release_debounce_job)
        self._release_debounce_job = self.after(10, self._actual_space_release)

    def _actual_space_release(self):
        self._release_debounce_job = None

        if self.state == 1:
            self.state = 0
            if self.history:
                self.lbl_time.configure(text=self.format_record(self.history[-1]), bootstyle=DEFAULT)
            else:
                self.lbl_time.configure(text="0.00", bootstyle=DEFAULT)

        if self.ready_job is not None: self.after_cancel(self.ready_job)

    elif self.state == 2:
        self.state = 3
        self.lbl_time.configure(bootstyle=DEFAULT)
        self.start_time = time.time()
        self._update_timer()

    def _update_timer(self):
        if self.state == 3:
            current = time.time() - self.start_time

```

```

self.lbl_time.configure(text=self.raw_format(current))
self.update_job = self.after(30, self._update_timer)

```

cl_parser.py

cl_parser.py

61 行

import re

class AdvancedParser:

@staticmethod

def inverse(alg_str):

"""求公式的逆运算"""

if not alg_str.strip(): return ""

moves = alg_str.split()

inv = []

for m in reversed(moves):

if m.endswith(" ") or m.endswith(","):

inv.append(m[:-1])

elif m.endswith("2"):

inv.append(m)

elif m.endswith("3"):

inv.append(m[:-1] + " ")

else:

inv.append(m + " ")

return " ".join(inv)

@classmethod

def parse(cls, alg):

"""核心解析方法"""

alg = alg.replace("'", "").replace(", ", ",").replace(": ", ":

")

while True:

1. 剥离圆括号 (...)

m_paren = re.search(r'\(((^()\[\]]+)\))(\d*|\')', alg)

if m_paren:

inner, mod = m_paren.group(1).strip(), m_paren.group(2)

res = cls.inverse(inner) if mod == "" else " ".join([i

nner] * int(mod)) if mod.isdigit() else inner

alg = alg[:m_paren.start()] + " " + res + " " + alg[m_p

aren.end():]

continue

2. 剥离方括号 [...] (交换子或共轭)

m_bracket = re.search(r'\[[(^()\[\]]+)\](\d*|\')', alg)

if m_bracket:

inner, mod = m_bracket.group(1).strip(), m_bracket.grou

p(2)

```

        if ':' in inner:
            setup, core = [p.strip() for p in inner.split(':',
1)]
            expanded = f"{setup} {core} {cls.inverse(setup)}"
        elif ',' in inner:
            parts = [p.strip() for p in inner.split(',', 1)]
            expanded = f"{parts[0]} {parts[1]} {cls.inverse(par
ts[0])} {cls.inverse(parts[1])}" if len(parts) == 2 else inner
        else:
            expanded = inner

        res = cls.inverse(expanded) if mod == "" else " ".join
([expanded] * int(mod)) if mod.isdigit() else expanded
        alg = alg[:m_bracket.start()] + " " + res + " " + alg[m
_bracket.end():]
        continue

# 3. 裸共軛 A: B
if ':' in alg:
    setup, core = [p.strip() for p in alg.split(':', 1)]
    alg = f"{setup} {core} {cls.inverse(setup)}"
    continue

break

return " ".join(alg.split())

```

completer.py

completer.py
149 行

```

import os
from concurrent.futures import ThreadPoolExecutor, as_completed
import cl_core

CORNER_FACELETS = [(8,9,20), (6,18,38), (0,36,47), (2,45,11), (29,26,1
5), (27,44,24), (33,53,42), (35,17,51)]
EDGE_FACELETS = [(5,10), (7,19), (3,37), (1,46), (32,16), (28,25), (3
0,43), (34,52), (23,12), (21,41), (50,39), (48,14)]
STD_CORNERS = [('U','R','F'), ('U','F','L'), ('U','L','B'), ('U','B','R
'), ('D','F','R'), ('D','L','F'), ('D','B','L'), ('D','R','B')]
STD_EDGES = [('U','R'), ('U','F'), ('U','L'), ('U','B'), ('D','R'),
('D','F'), ('D','L'), ('D','B'), ('F','R'), ('F','L'), ('B','L'), ('B',
'R')]

def _get_corner_rotations(c):
    return [c, (c[1], c[2], c[0]), (c[2], c[0], c[1])]

```

```

def _get_edge_rotations(e):
    return [e, (e[1], e[0])]

def _match_piece(target, piece):
    is_ignore_ori = any(t.islower() for t in target if t != '?')
    if is_ignore_ori:
        target_colors = set(t.upper() for t in target if t != '?')
        piece_colors = set(piece)
        return target_colors.issubset(piece_colors)
    else:
        for t, p in zip(target, piece):
            if t != '?' and t != p: return False
        return True

def generate_valid_completes(pseudo_str, stop_flag):
    c_targets = [tuple(pseudo_str[i] for i in idx) for idx in CORNER_FACELETS]
    e_targets = [tuple(pseudo_str[i] for i in idx) for idx in EDGE_FACELETS]
    valid_corners = []

    def dfs_corners(slot, used_mask, curr_cp, curr_co):
        if stop_flag(): return
        if slot == 8:
            if sum(curr_co) % 3 == 0:
                valid_corners.append((list(curr_cp), list(curr_co)))
            return
        target = c_targets[slot]
        for i, std_c in enumerate(STD_CORNERS):
            if not (used_mask & (1 << i)):
                for ori, rot_c in enumerate(_get_corner_rotations(std_c)):
                    if _match_piece(target, rot_c):
                        curr_cp.append(i); curr_co.append(ori)
                        dfs_corners(slot + 1, used_mask | (1 << i), curr_cp, curr_co)
                        curr_co.pop(); curr_cp.pop()

    dfs_corners(0, 0, [], [])
    if not valid_corners or stop_flag(): return

    def dfs_edges(slot, used_mask, curr_ep, curr_eo):
        if stop_flag(): return
        if slot == 12:
            if sum(curr_eo) % 2 == 0:
                ep_parity = cl_core.get_perm(curr_ep) % 2
                for cp, co in valid_corners:
                    if stop_flag(): return
                    if cl_core.get_perm(cp) % 2 == ep_parity:

```

```

        yield build_full_string(cp, co, curr_ep, curr_e
o)
        return
    target = e_targets[slot]
    for i, std_e in enumerate(STD_EDGES):
        if not (used_mask & (1 << i)):
            for ori, rot_e in enumerate(_get_edge_rotations(std_
e)):
                if _match_piece(target, rot_e):
                    curr_ep.append(i); curr_eo.append(ori)
                    yield from dfs_edges(slot + 1, used_mask | (1 <
< i), curr_ep, curr_eo)
                    curr_eo.pop(); curr_ep.pop()
            yield from dfs_edges(0, 0, [], [])

def build_full_string(cp, co, ep, eo):
    facelets = ['?'] * 54
    for i, c in enumerate([4, 13, 22, 31, 40, 49]):
        facelets[c] = ['U', 'R', 'F', 'D', 'L', 'B'][i]
    for i in range(8):
        for j, color in enumerate(_get_corner_rotations(STD_CORNERS[cp
[i]])[co[i]]):
            facelets[CORNER_FACELETS[i][j]] = color
    for i in range(12):
        for j, color in enumerate(_get_edge_rotations(STD_EDGES[ep[i]]
[eo[i]])):
            facelets[EDGE_FACELETS[i][j]] = color
    return "".join(facelets)

def solve_incomplete_stream(state_string, queue, stop_flag, search_mod
e, max_workers=0):
    """
    多线程并行残缺求解。
    利用 Cython nogil 释放 GIL 的特性, ThreadPoolExecutor
    可以让多个 CPU 核心同时运行 C 级搜索。
    """
    cl_core.init_engine()

    # 1. 收集所有物理合法的完整状态 (DFS 阶段, 纯 Python, 极快)
    all_states = []
    for comp_state in generate_valid_completes(state_string, stop fla
g):
        if stop_flag():
            break
        all_states.append(comp_state)

    if not all_states:
        queue.put(("DONE", 0))

```

```

        return

count = 0
seen_sols = set()

# 2. 定义单个状态的求解任务
def solve_single(state):
    """在线程池中执行—Cython 搜索时释放 GIL，真正并行。"""
    if stop_flag():
        return []
    results = []
    try:
        for sol in cl_core.solve(state, mode=search_mode,
                                max_depth=20, stop_flag=stop_flag):
            if stop_flag():
                break
            if sol != "":
                results.append((state, sol))
    except Exception:
        pass
    return results

# 3. 用线程池并行求解（线程数 = CPU 核心数）
if max_workers <= 0:
    max_workers = os.cpu_count() or 4
max_workers = min(max_workers, len(all_states))
with ThreadPoolExecutor(max_workers=max_workers) as executor:
    futures = {executor.submit(solve_single, st): st
               for st in all_states}
    try:
        for future in as_completed(futures):
            if stop_flag():
                # 取消尚未开始的任务
                for f in futures:
                    f.cancel()
                break
            try:
                results = future.result()
            except Exception as e:
                queue.put(("ERROR", str(e)))
                continue
            for comp_state, sol in results:
                if sol not in seen_sols:
                    seen_sols.add(sol)
                    count += 1
                    queue.put((count, comp_state, sol, len(sol.split())))
    except Exception as e:

```

```
queue.put(("ERROR", str(e)))

queue.put(("DONE", count))
```

locales.py

locales.py
232 行

locales.py

CURRENT_LANG = "en"

```
STRINGS = {
    "en": {
        "title": "Cube Library",
        "wait_task": "CL Engine initialized.\nWaiting for tasks...\n",
        "tab_facelet": "Facelet Editor",
        "lbl_selected_color": "Selected Color",
        "btn_customize": "Customize",
        "autofix": "AutoFix Colors",
        "grp_solver": "CL Solver Engine",
        "rad_twophase": "Two-Phase (Fast)",
        "rad_optimal": "Optimal (Slow)",
        "chk_infinite": "Infinite Mode",
        "btn_solve": "SOLVE CUBE",
        "grp_apply": "Apply Move (L-Click: 90°, R-Click: -90°)",
        "grp_reset": "Reset & Export",
        "btn_export": "Export String",
        "btn_empty": "Empty Cube",
        "btn_clean": "Clean Cube",
        "grp_maneuver": "Enter Maneuver",
        "btn_apply": "Apply",
        "btn_clear": "Clear",
        "menu_tools": "Tools",
        "menu_wca": "Generate WCA Scramble",
        "menu_lang": "Language",
        "menu_theme": "Theme",
        "menu_threads": "Solver Threads",
        "wait_scramble": "[Task] Generating Random WCA Scramble...",
        "copy_title": "Copied",
        "copy_msg": "Copied to clipboard:",
        "dlg_inc": "Incomplete Cube",
        "dlg_com": "Complete Cube",
        "dlg_title": "Infinite Search ({} ) - {}",
        "dlg_status_search": "Searching valid solutions...",
        "dlg_btn_stop": "Stop Search",
        "dlg_col_id": "#",
        "dlg_col_len": "Len",
        "dlg_col_sol": "Solution",
        "dlg_col_state": "Deduced State",
        "dlg_aborted": "Search Aborted by User.",
```

```

"dlg_btn_close": "Close",
"dlg_finished": "Search Finished! Found {} solutions.",
"dlg_found": "Searching... Found {} solutions so far.",
"tab_timer": "CL Training Timer",
"timer_loading": "Loading scramble...",
"timer_btn_delete": "Delete",
"timer_btn_next": "Next Scramble",
"timer_stats": "Solves: {} | Ao5: {} | Ao12: {}",
"timer_history": "History",
"timer_col_id": "#",
"timer_col_time": "Time",
"timer_btn_custom": "✎ Custom",
"timer_custom_title": "Custom Scramble",
"timer_custom_prompt": "Enter Custom Scramble:",
"timer_copy_title": "Copied",
"timer_copy_msg": "Scramble copied to clipboard!",
"timer_btn_sync": "🔄 Sync to Editor",
},
"zh": {
  "title": "魔方藏书室",
  "wait_task": "CL 引擎已初始化。\\n 等待任务输入...\\n",
  "tab_facelet": "展开图编辑器",
  "lbl_selected_color": "当前选中颜色",
  "btn_customize": "自定义配色",
  "autofix": "自动推导面色",
  "grp_solver": "CL 求解引擎",
  "rad_twophase": "双阶段模式 (极速)",
  "rad_optimal": "最少步模式 (最优)",
  "chk_infinite": "无尽求解模式",
  "btn_solve": "求 解 魔 方",
  "grp_apply": "应用转动 (左键: 90°, 右键: -90°)",
  "grp_reset": "重置与导出",
  "btn_export": "导出状态字符",
  "btn_empty": "清空魔方",
  "btn_clean": "复原标准态",
  "grp_maneuver": "输入自定义公式",
  "btn_apply": "执 行",
  "btn_clear": "清 空",
  "menu_tools": "实用工具",
  "menu_wca": "生成 WCA 随机打乱",
  "menu_lang": "语言 (Language)",
  "menu_theme": "界面主题",
  "menu_threads": "求解线程数",
  "wait_scramble": "[任务] 正在生成随机 WCA 状态打乱...",
  "copy_title": "已复制",
  "copy_msg": "已复制到剪贴板:",
  "dlg_inc": "残缺魔方",

```

```

"dlg_com": "完整魔方",
"dlg_title": "无尽搜索 ({} ) - {} ",
"dlg_status_search": "正在搜索有效解法...",
"dlg_btn_stop": "停 止",
"dlg_col_id": "序号",
"dlg_col_len": "步数",
"dlg_col_sol": "还原公式",
"dlg_col_state": "推导面色状态",
"dlg_aborted": "用户已终止搜索。",
"dlg_btn_close": "关 闭",
"dlg_finished": "搜索完成! 共找到 {} 个有效解。",
"dlg_found": "搜索中... 目前已找到 {} 个解法。",
"tab_timer": "CL 训练计时",
"timer_loading": "正在生成打乱...",
"timer_btn_delete": "删除该笔",
"timer_btn_next": "下一个打乱",
"timer_stats": "还原数: {} | Ao5: {} | Ao12: {} ",
"timer_history": "历史成绩",
"timer_col_id": "序号",
"timer_col_time": "成绩",
"timer_btn_custom": "✎ 自定义",
"timer_custom_title": "自定义打乱",
"timer_custom_prompt": "请输入自定义打乱公式:",
"timer_copy_title": "已复制",
"timer_copy_msg": "打乱公式已复制到剪贴板! ",
"timer_btn_sync": "🔄 同步到编辑器"
},
"zh_tw": {
"title": "魔術方塊藏書室",
"wait_task": "CL 引擎已初始化。\\n 等待任務輸入...\\n",
"tab_facelet": "展開圖編輯器",
"lbl_selected_color": "目前選取顏色",
"btn_customize": "自訂配色",
"autofix": "自動推導面色",
"grp_solver": "CL 求解引擎",
"rad_twophase": "雙階段模式 (極速)",
"rad_optimal": "最少步模式 (最佳)",
"chk_infinite": "無盡求解模式",
"btn_solve": "求 解 魔 方",
"grp_apply": "應用轉動 (左鍵: 90°, 右鍵: -90°)",
"grp_reset": "重置與匯出",
"btn_export": "匯出狀態字元",
"btn_empty": "清空魔方",
"btn_clean": "復原標準態",
"grp_maneuver": "輸入自訂公式",

```

```

"btn_apply": "執行",
"btn_clear": "清空",
"menu_tools": "實用工具",
"menu_wca": "產生 WCA 隨機打亂",
"menu_lang": "語言 (Language)",
"menu_theme": "介面主題",
"menu_threads": "求解執行緒數",
"wait_scramble": "[任務] 正在產生隨機 WCA 狀態打亂...",
"copy_title": "已複製",
"copy_msg": "已複製到剪貼簿:",
"dlg_inc": "殘缺魔方",
"dlg_com": "完整魔方",
"dlg_title": "無盡搜尋 ({} ) - {} ",
"dlg_status_search": "正在搜尋有效解法...",
"dlg_btn_stop": "停止",
"dlg_col_id": "序號",
"dlg_col_len": "步數",
"dlg_col_sol": "還原公式",
"dlg_col_state": "推導面色狀態",
"dlg_aborted": "使用者已終止搜尋。",
"dlg_btn_close": "關閉",
"dlg_finished": "搜尋完成! 共找到 {} 個有效解。",
"dlg_found": "搜尋中... 目前已找到 {} 個解法。",
"tab_timer": "CL 訓練計時",
"timer_loading": "正在產生打亂...",
"timer_btn_delete": "刪除該筆",
"timer_btn_next": "下一個打亂",
"timer_stats": "還原數: {} | Ao5: {} | Ao12: {} ",
"timer_history": "歷史成績",
"timer_col_id": "序號",
"timer_col_time": "成績",
"timer_btn_custom": "✎ 自訂",
"timer_custom_title": "自訂打亂",
"timer_custom_prompt": "請輸入自訂打亂公式:",
"timer_copy_title": "已複製",
"timer_copy_msg": "打亂公式已複製到剪貼簿!",
"timer_btn_sync": "🔄 同步到編輯器"
},
"pl": {
"title": "Cube Library",
"wait_task": "Silnik CL zainicjowany.\nCzekam na zadania...\n",
"tab_facelet": "Edytor Ścianek",
"lbl_selected_color": "Wybrany Kolor",
"btn_customize": "Dostosuj",
"autofix": "Autonaprawa Kolorów",
"grp_solver": "Silnik Rozwiązujący CL",

```

```

"rad_twophase": "Dwufazowy (Szybki)",
"rad_optimal": "Optymalny (Wolny)",
"chk_infinite": "Tryb Nieskończony",
"btn_solve": "ROZWIĄŻ KOSTKĘ",
"grp_apply": "Zastosuj Ruch (L-Klik: 90°, P-Klik: -90°)",
"grp_reset": "Resetuj i Eksportuj",
"btn_export": "Eksportuj Stan",
"btn_empty": "Wyczyść Kostkę",
"btn_clean": "Zresetuj Kostkę",
"grp_maneuver": "Wprowadź Manewr",
"btn_apply": "Zastosuj",
"btn_clear": "Wyczyść",
"menu_tools": "Narzędzia",
"menu_wca": "Generuj Pomieszczenie WCA",
"menu_lang": "Język (Language)",
"menu_theme": "Motyw",
"menu_threads": "Wątki solwera",
"wait_scramble": "[Zadanie] Generowanie losowego pomieszczenia WC
A...",
"copy_title": "Skopiowano",
"copy_msg": "Skopiowano do schowka:",
"dlg_inc": "Niekompletna Kostka",
"dlg_com": "Kompletna Kostka",
"dlg_title": "Nieskończone Szukanie ({} ) - {}",
"dlg_status_search": "Szukanie prawidłowych rozwiązań...",
"dlg_btn_stop": "Zatrzymaj",
"dlg_col_id": "#",
"dlg_col_len": "Ruchy",
"dlg_col_sol": "Rozwiązanie",
"dlg_col_state": "Wyprowadzony Stan",
"dlg_aborted": "Szukanie przerwane przez użytkownika.",
"dlg_btn_close": "Zamknij",
"dlg_finished": "Szukanie zakończone! Znaleziono {} rozwiązań.
",
"dlg_found": "Szukanie... Do tej pory znaleziono {} rozwiązań.
",
"tab_timer": "Stoper Treningowy",
"timer_loading": "Ładowanie pomieszczenia...",
"timer_btn_delete": "Usuń Ostatni",
"timer_btn_next": "Następne Pomieszczenie",
"timer_stats": "Rozwiązań: {} | Ao5: {} | Ao12: {} ",
"timer_history": "Historia",
"timer_col_id": "#",
"timer_col_time": "Czas",
"timer_btn_custom": "✎ Własne",
"timer_custom_title": "Własne Pomieszczenie",
"timer_custom_prompt": "Wprowadź Własne Pomieszczenie:",
"timer_copy_title": "Skopiowano",
"timer_copy_msg": "Pomieszczenie skopiowane do schowka!",
"timer_btn_sync": "🔄 Synchronizuj z edytorem"

```

```
}  
}
```

```
def tr(key):  
    return STRINGS.get(CURRENT_LANG, STRINGS["en"]).get(key, key)
```

setup.py

setup.py
19 行

```
"""
```

编译 Cython 搜索模块:

```
python setup.py build_ext --inplace  
"""
```

```
from setuptools import setup, Extension  
from Cython.Build import cythonize  
import numpy as np
```

```
setup(  
    name="cl_search",  
    ext_modules=cythonize([  
        Extension("cl_search", ["cl_search.pyx"],  
            include_dirs=[np.get_include()],  
            extra_compile_args=["-O3"])  
    ], compiler_directives={"language_level": "3",  
        "boundscheck": False,  
        "wraparound": False,  
        "cdivision": True}),  
)
```